ChatGPT

AI Tone Assistant – Technical Design

Technical Design Summary (Overview)

We propose an **"AI Tone Assistant"** that analyzes a guitarist's input and **recommends amp, cabinet (IR), and pedal settings** to emulate iconic heavy metal guitar tones. The system will support both **reference-based tone matching** (e.g. matching the sound of a given clip or artist/album tone) and **text-based tone descriptions** (e.g. "Gojira modern tight"). It uses a **standard high-gain signal chain** (Noise Gate → Boost/Overdrive → Amp → Cab/IR → EQ) with a normalized parameter schema, so that suggestions can be exported as presets for popular amp simulation software. The assistant functions as a **collaborative tool** – it quickly dials in a "ballpark" tone for the user to tweak further [1]. All AI recommendations are transparent and under user control (the user can A/B compare or adjust suggestions). The design emphasizes reliability with real guitarist inputs and **safety** (e.g. avoiding excessive volume or misleading matches).

At a high level, the system consists of:

- **Audio analysis** of the user's guitar input and optional reference track (extracting spectral and perceptual features).
- A **Tone matching engine** that uses a hybrid of **retrieval** (from a library of known tones) and **optimization** (iteratively adjusting parameters) to find the best settings matching the target tone.
- An **abstract TonePreset schema** representing the entire amp/effect chain in a vendor-neutral way. This preset can be translated to specific plugin formats or applied in a plugin host.
- **Integration components** for both offline analysis (e.g. loading a reference clip and computing suggestions) and optional real-time assist (low-latency monitoring to suggest minor tweaks on the fly).
- **Preset export utilities** to output the recommended settings as preset files for various VST amp sim plugins (or as a generic JSON if needed).
- A robust **evaluation and safety framework** including automated tests (feature similarity metrics, regression tests for "no fizz", etc.) and user listening tests (ABX comparisons) to ensure the assistant's recommendations are musically sound and not harmful (e.g. no sudden volume spikes).

This document details the design in ten sections: identifying user use-cases and constraints, defining the signal chain and preset schema, representing tone targets, audio feature extraction, core recommendation strategy, integration architecture, preset export formats, evaluation plan, risks & mitigations, and a decision log. We then provide an **architecture outline**, an **implementation roadmap**, and example **Codex prompts** to generate key components. Uncertainties are explicitly noted with proposed validation steps rather than speculation.

1. Users, Use Cases, and Constraints

Target Users: Electric guitarists (primarily intermediate to advanced) who use software amp simulators to achieve heavy metal tones. They may be recording at home, practicing, or performing live using plugin hosts. These users often seek to replicate famous guitar sounds or improve their own tone. They value tonal

authenticity (sounding like their metal heroes), but also ease-of-use (avoiding "option paralysis" when faced with countless amp and pedal settings [2] [3]). They are comfortable with desktop music software (DAWs, VST plugins) and likely own a selection of popular amp sim plugins.

Key Use Cases (prioritized):
1. **"Give me the `<Artist> - <Album/era>` rhythm tone."** The user specifies an iconic tone by artist/song/album or general label (e.g. *"Metallica – Master of Puppets tone"* or *"early 2000s Gothenburg melodeath rhythm sound"*). The assistant suggests a preset that emulates that signature tone. This caters to guitarists chasing famous sounds without knowing the exact gear.
2. **"Match this reference clip."** The user provides an audio sample of a guitar tone they want to mimic (e.g. a snippet from a song or a re-amped tone clip). The assistant analyzes the reference and the user's own DI (dry guitar) and produces amp/pedal settings so that when the user plays, it sounds as close as possible to the reference. This *tone matching* workflow is similar to existing tools like Fractal's Tone Match or Positive Grid's "Music-to-Tone" feature [4], aiming for spectral matching of the reference guitar sound.
3. **"Make my DI sound like a tight modern djent tone."** The user doesn't have a specific reference clip, but describes the desired tone characteristics (e.g. *"tight low end, percussive djent chugs with clarity"*). The assistant either uses the textual description or a generic exemplar of that style to recommend settings. This is a mix of text-based and learned style matching – matching the user's *playing* to an archetypal tone (even if the user's guitar/pickups might be different).
4. **"Suggest improvements to my current tone."** The user has dialed in a tone but finds issues (excess fizz, flubby bass, lack of bite, etc.). The assistant listens to the user's current tone (it could capture the processed output or evaluate settings) and suggests targeted adjustments. For example, *"reduce fizz by lowering treble or using a different IR,"* or *"add a tubescreamer pedal to tighten the low end."* This is a *refinement* use case, where the goal is not matching an external reference but optimizing tone quality against some ideal or user-specified goal.

Inputs and Context:
- **Guitar input type:** Primarily assume a **DI (direct input)** guitar signal (clean, unprocessed) from the user. The assistant can work with live input through an audio interface or with recorded DI tracks. In tone-matching scenarios, having the user's DI performance of the reference riff (or a similar riff) gives the best apples-to-apples comparison. The system should also handle an already **re-amped guitar track** (i.e. a recording that's been through an amp) if the user wants to match that sound – in this case the reference itself is the target tone.
- **Reference audio content:** Could be an isolated guitar track from a song, a mixed song segment (guitars + other instruments), or the user's own prior recording. **Constraint:** The reference should ideally be guitar-only or guitar-dominant. If the reference clip contains drums, bass, or vocals mixed in, the analysis might be confounded by those elements (see Risks). We may include a **source detection step** – if non-guitar content is detected or the spectral profile is clearly influenced by other instruments, the assistant either applies filtering or warns the user that a *reliable match requires an isolated guitar track*. Advanced versions might attempt *source separation* to extract the guitar from a mix, but this adds complexity and potential error (see Risk Mitigations).
- **Latency requirements:** For **offline analysis** (reference-based matching or text query), some processing latency is acceptable (a few seconds to perhaps 10-30 seconds at most for heavy analysis/optimization). This mode isn't real-time; the user can wait briefly for a suggestion after providing a reference or description. For **live assist** mode, latency must be minimal – ideally within a few milliseconds added to the normal audio path (i.e., not perceivably delaying the guitar signal). Live assist might only do lightweight continuous adjustments or analysis in small time slices, deferring heavy computations to moments of silence or just

providing *advisory visuals* to the user in real time (e.g. a meter showing "bass too high" but not actively changing the sound). **Target:** Real-time monitoring should remain below ~10ms additional latency and use limited CPU, whereas offline matching can use more CPU/GPU for a short duration (but likely still target <30s for a result on a typical PC).

Control Boundaries and User Agency:

- The assistant **only adjusts tone parameters** (amp EQ, gain, pedal on/off and settings, cabinet IR choice, etc.) – it does **not alter the user's playing performance** (notes, timing) or suggest changes to technique. It works *within the user's own playing and gear context*. For example, it won't tell the user "pick harder" or "change pickups" (though it might allow the user to inform it of what pickup is used for better accuracy).
- The user remains **in full control**. Any changes the assistant makes are presented as *suggestions* that the user can audition. A likely UX pattern is an **A/B comparison**: the user can toggle between their original tone and the AI-recommended tone to decide which they prefer or to fine-tune further. The assistant might present multiple options (e.g. "Option 1: closer match to reference's EQ; Option 2: slightly brighter version to cut through a mix") if appropriate.
- **Non-destructive operation:** The user's current preset or settings are not overwritten without consent. The assistant might create a *new preset* or a temporary state to demo the tone. Users should be able to easily revert to their initial sound.
- **Scope of adjustments:** By default, the assistant will adjust **gain structure, EQ settings (amp tone stack, presence, any active EQ blocks), cabinet IR selection, and pedal settings** (e.g. overdrive level, tone). It may also suggest adding or removing certain blocks: e.g., *"engage a Tube Screamer pedal in front of the amp"* or *"increase the noise gate threshold"*. However, it will not alter things like the user's actual guitar input level (other than advising if input is too hot/too low), and it won't affect playing style or notes. If the user specifically asks for performance advice (outside our current scope), we would treat that as out-of-scope for the Tone Assistant (that could be a separate feature entirely).
- **User preferences and constraints:** The assistant should respect any hard constraints the user sets. For example, a user might say "I only have Amplifier X plugin" or "Don't use any external boost pedal, just the amp gain". The system should allow the user to configure such constraints (perhaps via checkboxes like "Allowed to use overdrive pedal: Yes/No"). By default, it will assume all typical tone-shaping blocks are available to change. It should also consider practical limits – e.g., not suggesting *extreme* values that could produce unmusical results or equipment damage (like amp master volume at 100% causing output clipping, or insane treble boosts causing ear-fatigue).

UX Considerations:

- **Clarity and Explainability:** The assistant should ideally provide a short rationale for suggestions, teaching the user about tone. (For example: "Engaging a Tube Screamer pedal to tighten the low end and add mid boost, as is common for modern metal tones.") This aligns with Positive Grid's approach of having the AI explain its choices [5]. This is a nice-to-have that can build trust and help the user learn.
- **Easy iteration:** Users may want to refine the suggestions. The interface could offer tweak controls or follow-up prompts (e.g., "That's close, but still a bit too fizzy" – and the assistant can then adjust accordingly). We must ensure the design allows iterative cycles without becoming confusing.
- **Visual feedback:** If possible, graphically indicate changes (like knobs moving or frequency response comparison graphs) so the user can see what's being altered. Also, any differences between the reference and current tone could be visualized (e.g., a frequency response curve difference plot [6], highlighting where the current tone diverges from the target). This can again educate the user and let advanced users manually fine-tune if they prefer.

In summary, the Tone Assistant targets scenarios from *quick tone generation* (text or artist query) to *precise tone cloning* (reference matching), all under the principle that the guitarist remains the final judge. Key constraints like input type and latency modes are accounted for, and the user experience is designed to be helpful and not intrusive, keeping the user's creative control paramount.

2. Standard Guitar Signal Chain Model

To cover the vast majority of heavy metal guitar tones, we define a **standardized signal chain** and parameter taxonomy. This provides a consistent framework for analysis and preset suggestions, even if underlying plugins differ. The canonical chain is:

Noise Gate → Boost/Overdrive Pedal → Amp Head → Cabinet/IR → Post-EQ (optional)

- **Noise Gate:** The gate is placed first to suppress input noise (hum, pickup buzz) during pauses. Key parameters include **Threshold** (dB level at which gating kicks in), **Release** (how quickly it opens/closes), and possibly **Attack**. For tone matching, the gate mostly affects the feel and silence between notes rather than frequency content, but for *"tightness"* in modern metal (staccato palm-mutes), a properly set gate is important. We allow the assistant to suggest threshold adjustments or turning the gate on/off, but not a drastic change like removing it if the user clearly needs one (heavy high-gain always needs noise control).

- **Boost/Overdrive Pedal:** Typically an overdrive (like an Ibanez Tube Screamer or Maxon OD808) is placed before the amp to tighten the low-end and add saturation. Parameters: **Drive (Gain)**, **Tone** (which is effectively a high-pass or mid hump tone control on many pedals), and **Level** (output level into the amp). The assistant can toggle adding a boost if the target tone calls for it (e.g. most modern metal tones use a TS-style boost to cut bass and push mids). If the user's current tone lacks one and the target is a djent tone, it will likely recommend engaging one. We also allow different pedal models (the schema might have a "pedal type" field), though in high-level terms, most boosts serve similar purpose – the main difference is their tone shaping (some are more mid-focused, some have more bass cut). For normalization, we represent pedal controls in **unit ranges** (0.0 to 1.0) internally, mapping to specific pedals: e.g., Drive 0.0 = minimum drive, 1.0 = max; Tone 0.5 = noon, etc., even if different pedals label knobs differently.

- **Amplifier Head (Preamp + Power Amp):** The amp provides the core distortion and tone shaping. We consider various *amp models*, from classic Marshall and Peavey 5150 variants to modern Mesa Boogie, ENGL, etc., as these define the character of the tone. Key parameters: **Gain** (preamp gain), **Bass, Mid, Treble** (tone stack EQ), **Presence** and **Depth/Resonance** (power amp feedback controls for high and low end respectively, if available), **Master Volume** (which in some amps affects power amp saturation). Some amps may have additional switches (e.g. bright switch, voicing switches). The **TonePreset schema** will include the amp *model identifier* (e.g. "EVH 5150 III") and a mapping of all relevant knob settings. We will normalize these controls across different amp models: for example, represent **tone stack EQ in dB or 0–10 scale** regardless of plugin, and **presence/depth** also on a 0–10 or 0–100% scale. We are aware that different amp models have different tapering for controls (e.g., "Presence at 5" on one amp might be much brighter than "Presence at 5" on another). Our mapping system will store model-specific calibration if needed – for instance, we could measure each amp model's frequency response change per knob increment offline and use that to translate a desired frequency profile adjustment into the appropriate knob change for that model. This way, the

assistant's logic can think in terms of *"increase high-frequency presence by X dB"* and map that to the correct control move on whichever amp is in use.

- **Cabinet and IR (Impulse Response):** The cabinet (speaker + mic setup) is arguably as important as the amp for final tone. We represent this stage as either a **Cabinet model (if using built-in cab sim)** or an **IR file**. Most modern workflows use IRs – essentially WAV files capturing the speaker+mic frequency response. In our schema, the **IR selection is categorical** but with rich metadata: e.g., *Cabinet: Marshall 1960 4x12, Speakers: Celestion V30, Mic: Shure SM57 on-axis*. We'll have a way to identify IRs either by a unique ID if from a known library or by these attributes. For matching, the assistant might say "use a Mesa Oversized 4x12 with V30s and an SM57 mic position" if the target tone matches that profile. If the user's amp sim plugin has a built-in IR loader, we can load a specific IR file in it. If the plugin uses its own cab models, we map the suggestion to the closest available cab model (for example, in BIAS FX or AmpliTube, pick the cab that best fits the description). **Normalization:** We consider the IR as a block with no continuously variable parameters except maybe a **high-cut filter** or mic position simulation if the plugin supports it. But typically, we treat the IR choice as discrete. We maintain a list of canonical cab types and associate each with representative IR files. The TonePreset schema might have fields like `cabinet_model` (string or enum), `mic_type`, `mic_position`, and an `IR_file` reference if applicable.

- **Post-EQ (Optional):** Many produced guitar tones have an EQ after the cab to fine-tune the spectrum (for example, cutting some honky mids or taming fizz above 8kHz). We include an optional **post-EQ block** (could be parametric or graphic EQ). Key parameters would be the **EQ curve** (which frequencies are cut/boosted and by how much). Representing an arbitrary EQ curve is complex; we could limit to a few typical bands or have a flexible representation (list of bands with frequency, gain, Q). For the TonePreset, we might include something like an array of `{freq, gain_db, Q}` for each band if an EQ is used. If the target tone requires precise matching, the assistant might employ a matching EQ curve here (this is analogous to how Fractal's Tone Match or Kemper profiling essentially produce an IR or EQ to match the frequency response [6] [7]). We must be cautious: relying on a custom post-EQ too heavily might make the preset less transferable to other amp sims (since not all will have the exact same EQ capability). In practice, we can try to approximate needed EQ tweaks using the amp's controls and IR choice first, and reserve post-EQ for small refinements that any major amp sim would allow (most have some post-EQ or the user can add one).

Parameter Taxonomy and Normalization: Every parameter in the chain will be represented in a consistent **TonePreset schema**. We use SI units or standardized scales where possible: e.g., frequencies in Hz, gains in dB or on 0–10 scale if relative, times in milliseconds, etc. For continuous controls like knob positions, we'll generally use a 0.0–1.0 normalized value internally (and possibly also store the original "unit" like 7/10 or 50%). Categorical choices (amp model, pedal type, cab type) will use defined identifiers. This abstraction is critical for mapping to multiple plugin vendors and for the AI to reason about settings agnostic to a specific plugin.

- **Example TonePreset schema snippet:** For illustration, a JSON-like representation might be:

```
{
  "noise_gate": {"threshold_db": -40.0, "release_ms": 100},
  "pedal": {"model": "TubeScreamer TS9", "drive": 0.2, "tone": 0.6, "level":
0.8, "enabled": true},
```

```
  "amp": {"model": "Peavey 5150", "gain": 0.85, "bass": 0.4, "mid": 0.5,
"treble": 0.6, "presence": 0.7, "depth": 0.6, "master": 0.5},
  "cabinet": {"type": "Mesa Oversize 4x12 V30", "mic": "SM57_on_axis",
"IR_file": "mesa_v30_sm57.wav"},
  "post_eq": {"bands": [ { "freq_hz": 8000, "gain_db": -3.0, "Q": 1.0 } ]}
}
```

This indicates we have a TS9 boost (low drive, high level), a 5150 amp with certain knob settings, a Mesa/V30 cab IR mic'd with SM57, and a post-EQ cutting 8 kHz by 3 dB (to tame fizz). This abstract preset can be later translated to specific plugin parameters (see Preset Export section).

Consistency Across Plugins: We will maintain a mapping of this schema to each supported plugin's parameters. For example, "Presence" in our schema might map to "Hi Contour" in some amp plugin if named differently. If a particular amp model lacks a parameter (e.g., some amps don't have a Depth knob), we simply leave it default or ignore it for that model. If an amp sim has extra parameters not in our schema (e.g., tube bias, advanced tweaks), the assistant will not utilize them (or will leave them at default) unless we explicitly extend the schema. This keeps the scope focused on primary tone controls that have the biggest impact.

By defining this standard chain and schema, the Tone Assistant can reason about tone in a **vendor-neutral** way. It can say conceptually "we need a high-gain British amp with a mid scoop and a V30 cab" and then instantiate that on (for example) **Neural DSP's Fortin Cali plugin** or **Line 6 Helix Native** by picking the corresponding amp/cab models and dialing the normalized values. This also makes it easier to store and compare tones across a library, and to perform algorithmic optimizations regardless of which specific gear is used under the hood. All in all, the standardized chain is a **foundation for both the AI's internal logic and the preset interoperability**.

3. Tone Target Representation

Defining how we represent the "target tone" is crucial, since the assistant can accept different kinds of targets. We consider and combine multiple approaches:

- **Reference Audio Based (Direct Analysis):** In this mode, the target tone is defined by an **audio sample**. The system will extract a set of features or a "fingerprint" from the reference guitar tone. This could include the spectral shape (EQ curve), distortion characteristics, noise profile, etc. Essentially, the reference gives us an *acoustic target* that we try to match. This approach is concrete: we measure how the user's processed guitar would need to sound (in measurable features) to match the reference. For example, if the reference has a big dip at 500 Hz and a spike at 1200 Hz in the spectrum, the assistant will try to replicate that EQ difference via amp/pedal settings or post-EQ. Reference-based matching is powerful for capturing subtle nuances of a specific recording. It's used in products like Fractal's Tone Match which uses an FFT of reference vs local tone to generate a matching filter [6]. We plan to use an extended version of that idea: not just an EQ match, but adjusting distortion and other factors as well. We need to clearly define *what "match" means* in operational terms here: likely a combination of **spectral similarity** (the frequency response of the user's tone vs reference should be as close as possible) and **dynamic similarity** (the character of how notes distort, sustain, etc.). We will formalize this in section 4 with metrics.

- *Output of analysis:* The reference analysis could yield a target **spectral envelope** (e.g. an average frequency response curve of the reference guitar tone), target amounts of high-frequency "fizz", low-end boom, etc., and possibly a target **distortion profile** (like an estimate of how compressed or saturated the reference is). These become goals for the matching algorithm. Because a raw FFT comparison can be too naive (it might try to match unrelated aspects like specific pick noise in the reference), we will focus on robust features (averaged or statistical descriptors over the clip).

- **Text/Label Based (Descriptive):** In this scenario, the user provides a **description or label** of the tone. This could be very direct (naming an artist/album or amp model) or adjectival (e.g. "tight, scooped, brutal, with a lot of presence"). The system needs to map this language to a target tone representation. One approach is to have a predefined mapping from known references: e.g., *"Master of Puppets tone"* could internally link to known gear settings (Mesa Mark IIC+ amp, V30 cab, scooped mids) or even a reference IR of that tone. Another approach is using a **machine learning model** or embedding that connects text and audio (for example, using something like a CLIP model but for audio, or a text-to-audio retrieval model). For the initial implementation, a pragmatic way is to maintain a library of iconic tones labeled with names, and/or rules: e.g., "if user says 'scooped thrash tone', target a 80s Marshall or Mesa with bass/treble high, mids low." We can also incorporate known **artist gear metadata**: for instance, if user asks for *"Gojira's tone"*, we know Gojira often use Peavey/EVH 5150 amps with Tube Screamer and a Mesa cab; that informs our starting point.

- *Operational definition:* For text input, *"match" means to produce a tone that users familiar with that description would agree is in that style*. This is inherently subjective, so success is measured by user satisfaction (or perhaps by expert rating in evaluation). We can still break it down into features: e.g., "thrash scooped" implies a certain EQ curve (mids cut), "djent tight" implies very fast low-frequency response and high mid presence, etc. We will encode these target characteristics either via templates or by training on example tones labeled with those descriptors.

- **Hybrid (Combined Input):** The most powerful usage might be combining both: e.g., the user says "I want a Gojira-like tone" *and* provides their own DI riff. Here the system can use the text to pick an appropriate reference tone profile (maybe load a known Gojira preset or profile as a starting point), and then fine-tune it to the user's DI characteristics or further audio goals. Another hybrid scenario is **text + reference**: user could supply a reference track plus say "make it a bit brighter than this" – mixing objective reference with subjective instruction. Our system architecture will treat any available input as constraints or goals that can be weighed. We can give precedence to reference audio if provided (since it's concrete), but still use text to handle aspects the audio might not cover (like "I want X tone but with slightly less gain").

Defining "Match" Criteria:
We avoid vagueness by specifying metrics for matching a tone. These metrics will guide the optimization/search algorithm and also our evaluation. For a given target (reference or textual ideal), we define a **multi-objective similarity score** composed of:

- **Spectral Similarity:** e.g., the difference in frequency response between the user's tone and target. We can quantify this by computing an average spectrum for both (perhaps in critical bands or Mel bands) and measuring error (like an L2 norm of the difference, or a weighted sum of differences in specific bands). We might use a perceptual weighting (so differences in the mid frequencies are treated as more important for tone character than differences at extreme highs or lows beyond

guitar range). A “perfect” match would have essentially the same spectral shape. However, we consider trade-offs: an exact spectral match might be possible by an FIR filter (like how pure tone-match just applies an IR), but that could mask differences in distortion texture or noise. So spectral match is necessary but not sufficient.

- **Distortion/Dynamics Match:** Heavy guitar tones differ in how saturated or compressed they are. We can measure something like the **harmonic content** (e.g., via calculating a spectral flatness or high-order harmonic energy – a heavily distorted tone will have more high-order harmonics and a more compressed dynamic range). We might define a metric like *“dynamic range compression difference”* – comparing, say, the crest factor (peak vs RMS level) of the user tone vs reference. Another measure: the **Transient decay** – e.g., how quickly a palm-muted note’s amplitude falls off, which can be influenced by gain and gating. If the target is a very percussive djent tone, the transient might be a sharper spike followed by quick damping (which a gate or high damping can cause). The assistant should match that envelope shape. We could quantify this by analyzing the envelope of low-frequency content of palm mutes in both signals and comparing decay time constants.

- **Noise floor and gating:** If the reference has near-silence between riffs (indicating aggressive noise gating), the user’s tone should replicate that (which implies setting a similar gate threshold). If the reference is a studio track with no noise at all, we should ensure our preset doesn’t introduce audible noise (which might mean recommending a gate or adjusting one). We can measure noise floor by looking at segments of the reference where no guitar is playing (if provided) vs the user’s chain idle noise, and attempt to match or at least not exceed it.

- **Time-domain characteristics and feel:** These are harder to measure but might include *pick attack clarity* (how distinct the initial pick transient is). Potential metric: measure the high-frequency content in the first ~20ms of notes (a bright, articulate pick attack will show a spike in high frequencies on attack). Another metric could be *inter-note consistency*: e.g., in fast palm-muted passages, if the reference has very even, machine-gun like chugs (often from compression), our tone should produce similarly even amplitude chugs. We might use statistical measures on note segments (like variance in amplitude between palm mute hits) – though this also depends on playing technique, which we assume similar if user plays the same riff.

In summary, we define the “matching objective” as a weighted combination of these factors: *spectral difference*, *dynamic/transient difference*, *noise difference*, etc. For text-based targets, we don’t have a concrete reference to compute these, so instead we use either a proxy reference (from our library) or predetermined target feature values. For example, for “tight modern djent”, we might specify target features like: spectral tilt slightly towards high-mids, very low sub-bass, very short low-frequency decay, high gating, moderate-high distortion. These become target numbers that the algorithm tries to achieve when processing the user’s DI through the candidate settings.

One crucial aspect: **loudness normalization.** Before comparing spectra or dynamics, we should ensure both signals (user’s and reference) are at comparable loudness. A louder signal will naturally have a different spectrum perception (e.g. equal-loudness contours). We likely normalize both to a reference level (say -14 LUFS or similar) or at least match RMS level in a given band when computing differences. The assistant’s output preset should be loudness-matched to the reference as well, to avoid psychoacoustic bias (see Safety: we also avoid boosting level too far).

Trade-offs: In practice, an exact match on all metrics might be impossible, especially if the user's guitar/pickup is different. For instance, if the user's guitar is inherently brighter, even with the same amp settings it might sound brighter than the reference. The system will attempt to compensate (maybe reduce treble), but only up to a point – we cannot fully negate the instrument's character. So "match" also means *coming as close as feasible given the circumstances*. If conflicts arise (e.g. matching the reference spectrum would require an extremely unusual amp setting that introduces other problems), we will prioritize overall perceptual similarity over literal knob parity. The user ultimately cares that it *sounds* right, not that "Bass=4 vs 6". So the assistant might produce an approximation that in isolation isn't identical to the reference, but in context (especially if the user plays along the reference) it blends.

Conclusion: The tone target representation can be thought of as either a **target feature vector** (for audio-based targets) or a **semantic target vector** (for text targets). Our system will unify these by possibly converting text queries to a pseudo "ideal audio features" representation via a library or model. During matching, we continuously evaluate how close we are to that target representation using the defined metrics. In the next section, we detail what features are extracted to facilitate this comparison robustly.

4. Audio Feature Extraction and Metrics

To compare and match tones, the assistant will extract a comprehensive set of audio features from both the **reference clip** (if provided) and the **user's guitar output** (through a candidate preset). These features should be robust to differences in playing (e.g. different notes or riffs) and capture perceptually salient aspects of guitar tone. Below we outline key features and how they contribute:

Spectral Features:
- **Spectral Centroid:** Represents the "brightness" of the tone – essentially the weighted average frequency. A higher centroid means a brighter, more treble-heavy tone [8]. Distorted metal guitars often have a centroid in the mid to high frequency range due to strong upper harmonics. This metric helps ensure we match the overall brightness/darkness of the tone.
- **Spectral Tilt / Slope:** The overall tilt of the frequency response from low to high. For instance, a thrash metal "scooped" tone might have strong bass and treble but dip in mids, which would show a certain tilt shape (perhaps a U-shaped EQ rather than a straight slope). We can compute a linear regression on the log-spectrum to get a slope value or use low vs high band energy ratios to quantify if the tone is bass-heavy, mid-scooped, etc.
- **Band Energy Ratios:** We will likely define specific frequency bands of interest for guitar: - Low-end (e.g. 60–250 Hz): Too much here can indicate "mud" or flabbiness; too little can indicate a thin tone. - Low-mid (250–500 Hz): Excess here can sound "boxy" or "woody". - Mid (500 Hz–1.5 kHz): The body of the guitar tone; metal rhythm often scoops around 500 Hz for chunk, but if scooped too far, clarity of notes (esp. on smaller speakers) suffers. - High-mid (1.5–4 kHz): Critical for attack and presence; too much can be "ice-pick" painful on ears. - Treble "fizz" region (4–8 kHz, and 8–12 kHz): These upper bands contribute to the sizzle of distortion. A controlled amount gives air and definition, but too much is undesirable fizz/hiss. We will extract energy or average level in each band for both signals. The target matching will try to minimize the difference in each band (possibly weighted by importance). For example, if the reference has -10 dB in the 500 Hz band relative to other bands, and the user's current tone doesn't, the assistant will see a mismatch and adjust (likely lowering mids on the amp or adding a scoop EQ). These band-wise comparisons are essentially how traditional EQ matching works – flattening the difference curve [6].
- **Harmonic Content / Distortion Profile:** We can perform a harmonic analysis by feeding a known test tone or looking at sustained notes in the reference. Distortion generates harmonics; the distribution of

those (especially odd vs even harmonics) can differentiate say a fuzz vs a tight distortion. However, doing this on a full riff is complex. Instead, we might use **Spectral Flatness** or **Odd-harmonic ratio** as features: - *Spectral Flatness* (per band): measures how noise-like vs tone-like a signal is. A fully distorted guitar is noise-like across much of the spectrum (flatness closer to 1) except maybe fundamental frequencies. A lightly overdriven guitar is more tonal (flatness < 1). We can compare the reference and user output flatness to gauge if our distortion amount is similar. - *Inharmonicity:* Distortion should produce harmonics at integer multiples of the fundamental notes. If we detect a lot of non-harmonic content (like weird whistling frequencies), that's a sign of artifacts (maybe too much fizz or a bad IR resonance). We aim to minimize unexplained spikes in the spectrum not present in reference.

Time-Domain / Dynamic Features:

- **Transient Attack Strength:** We can detect note onsets (using an onset detector or simply threshold crossing on high-frequency content) and measure the amplitude envelope around them. A sharp attack will have a high initial peak relative to the sustain. If the reference tone has clearly defined pick attacks, we want the user's tone to preserve that. For instance, if a compressor or too much gain is smearing the attack, our feature would show a lower attack-to-sustain ratio. We want to match the reference's ratio by adjusting gain or recommending less compression if needed.
- **Palm-mute Tightness (Low-end damping):** For heavy palm-muted chugs, the *tightness* is often defined by how quickly the low-frequency energy decays after the initial hit. A loose tone might let the low frequencies ring out or swell (boominess), whereas a tight tone (often aided by a tubescreamer and high damping in the amp) will cut the low frequencies quickly. We can isolate the low-frequency band (say <150 Hz) and examine the waveform envelope for each palm mute (the reference vs user's). We could compute something like the **decay time to drop 60 dB** or simply the energy ratio of first 100ms vs next 200ms. Matching that ratio/duration can quantify tightness. Another indicator: the **peak-to-average ratio in low frequencies** on palm mutes – tight tones have a high peak (initial chug) and then it stops, so average is much lower; loose tones have more sustained low-frequency energy. We can derive a metric from that.
- **Inter-note Noise and Gating:** If possible, detect segments where the guitar isn't playing (silences or rests). Measure the noise level there for both reference and user's chain. If the user's chain noise is higher, and if that matters for target, the assistant will tighten the gate or advise noise reduction. Conversely, if the reference has some natural amp noise (some classic recordings do), we might not completely dead-gate the user's tone as it could feel unnatural – but usually, quieter is better. We may set a threshold that the noise floor of the user's tone should be at or below the reference's.

Handling Different Riffs / Content Normalization:

One challenge is that the reference and user's playing might not be identical phrases. A straightforward spectral comparison could be thrown off by that (e.g., if the reference plays a lot of open low chugs and the user's test is playing higher chords, the spectra will differ due to musical content, not just tone). To mitigate this:

- We can ask the user to play a **similar riff** for the analysis if possible (the UI could encourage playing the same notes/riff as the reference if they're trying to match a specific clip). In absence of that, we will try to **segment and classify content**: - Identify sections in each audio: e.g., find all palm-mute chug sections, find all sustained chords, find single-note lines, etc. Then compare like with like. If the reference has a section of chugs, and the user's recording also has some chugs, compare the spectral content during those chugs specifically. This requires some intelligent segmentation (onset detection, power in low frequencies can indicate chugs, etc.). We might use simple rules or even a lightweight ML model to classify segments of playing (percussive palm mutes vs ringing chords vs lead notes).
- If the user's input lacks a type of playing present in reference (or vice versa), focus on the common

elements or the overall average. We just need to be careful not to penalize differences that are purely performance. For example, a squealy high harmonic in the reference might show high frequencies that the user's recording with no such squeal won't have – our system should ignore that outlier or consider band energy only during comparable times.
- We can also incorporate **instrument-independent embeddings**: There are research models (like OpenL3 or others) that create embeddings of audio that focus on timbral qualities rather than exact pitch content. Using such an embedding (trained perhaps on identifying audio production style [9]) could help measure similarity in tone while being somewhat invariant to the actual notes played. For MVP, this might be overkill, but it's an avenue (for instance, the ST-ITO method trains an audio representation specifically to capture production style and uses that as the metric [10] [11]). We may initially rely on our handcrafted features and later validate with a learned metric.

Similarity Metric and Improvement Metric:
All these features will feed into a **scoring function**. For matching a reference, we'll calculate a similarity score S where higher means "sounds closer to target." S could be composed as:

$$S = w_{\text{spec}} * (-\text{SpectralError}) + w_{\text{dyn}} * (-\text{DynamicsError}) + w_{\text{noise}} * (-\text{NoiseFloorDiff}) + \dots$$

Essentially subtracting errors weighted by importance (with possibly some nonlinear penalties for certain things like any huge spike in 4kHz might be heavily penalized as "ice pick" even if overall spectrum matches). The weights might be tuned from experiments or listening tests (e.g., spectral match might get 50% weight, dynamics 30%, etc., reflecting that tone is first about EQ, then about feel).

For the *improvement use-case* (user just says "reduce fizz" or generally "improve my tone"), we can define an ideal target in terms of avoiding known bad extremes: - For example, we have thresholds: no frequency band should be more than X dB above a flat reference, high-end (e.g. 8kHz+) energy should be under a certain ratio relative to midrange (to avoid fizz), dynamic range should be within a certain window (not too compressed to lose all punch, and not too dynamic if they want a sustained metal tone). We effectively define a **desired range** for each feature (based on genre norms or user preference). The assistant then acts to bring the tone into those ranges. This is less precise than matching a specific reference but uses the same feature framework to quantify "fizz" (excess high frequency energy) or "mud" (excess <100Hz energy, or too slow a low-end decay).
- We will implement specific detectors: e.g., an **"ice-pick detector"** might check if there's a narrow high-mid frequency (3–5 kHz) that's overly pronounced; a **"over-gain detector"** might check if spectral flatness is too close to 1 across all frequencies (indicating maybe too much saturation making everything a square wave). These can be part of evaluation and safety, but also inform improvement suggestions.

Summary of Feature Pipeline:
1. **Preprocessing:** Loudness normalize and perhaps band-limit to focus on guitar range (e.g. 50 Hz – 12 kHz) to avoid influence of irrelevant frequencies. Possibly apply a windowing to exclude silence or count it separately as noise.
2. **Compute spectral descriptors:** overall spectrum, critical band energies, centroid, etc., from a long-term average and maybe also short-term frames.
3. **Compute dynamic descriptors:** envelope stats for transients and sustains, crest factor, note decay constants.
4. **Compute specialized measures:** if needed, things like spectral flatness (maybe in sub-bands), or use an ML embedding for a sanity check.

5. **Normalize features for content differences:** e.g., might average metrics per note or per segment type as described, to reduce dependency on particular riff content.
6. **Comparison:** subtract or ratio the user's vs target's feature values to get "differences."
7. **Aggregate into scores or feed to optimizer.**

These features and metrics will be used in two places: (a) **inside the recommendation engine** (the optimization or model will use them to judge how well a candidate setting is doing), and (b) **in evaluation** (we'll use the same metrics to automatically test how close the final suggestion got, and to flag any problematic tonal issues).

By carefully choosing robust features, we aim to ensure the assistant isn't "fooled" by irrelevant factors (like different riffs or slight performance nuances) and truly captures the tonal character differences that matter to guitarists (fizz, tightness, clarity, etc.). In the next section, we discuss how these features are used in strategies to actually find the best amp/pedal settings – essentially the "intelligence" of the system.

5. Recommendation Strategies (Core Intelligence)

The core task is to determine **amp/pedal/IR settings that achieve the target tone** as represented by the features above. We evaluate several strategy paradigms and choose a hybrid approach:

A. Search/Optimization in Parameter Space:
One approach is to treat the tone-matching as an optimization problem: define an objective function (based on the similarity metrics in section 4) and search the **space of possible parameter settings** for the combination that maximizes similarity to the target. This is essentially a "black-box" optimization since the relationship from parameters to sound is complex and often non-differentiable (especially if using actual plugin DSP). Techniques for this include: - **Evolutionary Algorithms (EA):** e.g., **CMA-ES (Covariance Matrix Adaptation Evolution Strategy)** or genetic algorithms. These can handle continuous parameters and don't require gradients. We could randomly initialize a population of parameter sets (perhaps around a sensible guess) and then iteratively evolve them by keeping those that sound closer to target. CMA-ES is often good for continuous optimization in moderate dimensions (our parameter space might be ~10-15 dimensions if we include a few knobs and pedal on/off as discrete). It can find a good solution in relatively few iterations if the objective landscape isn't too deceptive. - **Bayesian Optimization:** This would treat the plugin as a black box that we sample; a Gaussian process or similar tries parameter sets and models the objective to propose the next best query. This can be efficient if objective evaluations are expensive. However, high dimensional and potentially multi-modal landscapes may be challenging. - **Brute-force or grid search in limited subspace:** For certain sub-problems, we might do a coarse grid. For example, matching an EQ curve could involve trying a few mid knob positions and seeing effect. But brute force for all combos is infeasible beyond a very low dimension. - **Gradient-based (differentiable proxies):** Since most amp plugins are not differentiable, true gradient descent on the actual audio difference isn't possible. But one can train a *neural surrogate model* of the amp (a differentiable approximation) and then use gradients on that. There have been research efforts like differentiable IIR filters or using neural nets to approximate effects [12] [13], but implementing that for arbitrary third-party plugins is non-trivial. We likely skip direct gradients and stick to gradient-free search or possibly use **finite-difference gradient approximation** on small subsets if needed (which is essentially still black-box sampling).

Pros: Optimization can fine-tune to very precise matches, potentially achieving extremely close results (especially in spectral matching). It is flexible – it can adjust any parameter, including non-intuitive

combinations, to meet the goal. It doesn't require a training dataset of sounds; it works on the fly by trial and error, guided by the objective. For instance, **ST-ITO (Style Transfer with Inference-Time Optimization)** uses exactly this idea of iteratively searching effect parameters to match a reference style [10] [11]. It proved effective even for unseen effects, using gradient-free optimization with a learned similarity metric.

Cons: Pure optimization can be computationally heavy – each evaluation means processing audio through the effect chain and computing features. If we need dozens or hundreds of iterations, that could mean seconds or even minutes of processing, which might be too slow for a user-facing tool. Also, black-box optimization might get stuck in local minima or produce oddball solutions if the objective isn't perfectly capturing perceptual quality (e.g., it might over-optimize one aspect at expense of another if not weighted properly). Additionally, some parameters are discrete (like which amp model to choose, or turning a pedal on/off); optimization algorithms handle continuous ones well, but discrete choices might require special handling (we might treat model selection via a separate outer loop or use heuristics).

Given these, **we will include an optimization engine** but likely not rely on it as the first step. Instead, we can use it for *refinement*: start from a good guess and let the optimizer polish the settings to closer match. This yields the best of both worlds – it doesn't have to search blindly across the entire space.

B. Retrieval from Curated Tone Library:
Another approach is to leverage existing knowledge: have a database of **known good presets and tones** (with their audio characteristics indexed), and simply find the closest match to the target tone in that library. This is essentially a nearest-neighbor search in *tone space*. The library could include: - Presets modeled after famous albums or artists (e.g., a preset for "Master of Puppets", one for "Dimebag Darrell tone", etc.). - Presets covering typical archetypes (e.g., "Modern Djent 8-string tone", "80s thrash tone", "Swedish death metal HM-2 buzzsaw tone"). - The library could be built in-house or partly crowdsourced (users share tones labeled with styles). Initially, we'd likely curate a set from known free presets or by manually dialing in and labeling some. - The library entries would each have a TonePreset (parameters) and a precomputed feature vector (using the same extraction as we do for targets).

When a user provides a target (text or audio), we query this library: - For text queries, we can map the text to either a set of candidate library entries by tag matching (e.g., if query contains "Metallica", library entry "Metallica MoP preset" is obvious first pick) or by using a text embedding if we have one for the preset's description. But a simpler approach: we can assign tags to each library preset and do an overlap with query keywords. - For audio queries, we compute the features of the reference and then find the preset(s) with most similar feature vectors (like nearest neighbor in a weighted Euclidean space or via some distance metric). - The top result from retrieval can be used *directly* as a suggestion if it's very close, or as a **starting point** for further optimization. For example, if the user references a tone that is basically an Engl amp sound and our library has an Engl preset with similar EQ, retrieval will catch that quickly. We then might just fine-tune a knob or two.

Pros: Retrieval is extremely fast once the library is built (just feature comparisons). It also injects human knowledge because those presets are presumably well-crafted. It avoids weird combinations that pure algorithm might try, and ensures a *musically sensible baseline*. If the library is large enough and covers many styles, retrieval could handle the majority of cases by giving a "90% correct" solution immediately. This is how some tone assistant prototypes might work – essentially a fancy preset browser powered by similarity search. It's also easier to implement the text interface via retrieval: if user says "I want Lamb of God tone", we directly fetch a preset labeled "Lamb of God" if available.

Cons: The library is limited. There will be tones not covered or new user preferences that don't match exactly. If the query is something niche or the user's guitar/pickup is very different, the library preset might not be optimal (e.g. library tone was made with an ESP with EMGs, but user has a Strat single coil – the same preset might not yield the same sound on their guitar). We can mitigate by having multiple versions (perhaps calibrate library tones with different guitar DI sources). There's also maintenance – adding new tones over time, ensuring quality, etc. Additionally, reliance on retrieval alone might produce suggestions that are "closest in library" but still audibly off; hence, combining with refinement is prudent.

We will **build a tone library dataset** to enable retrieval. **Data collection methods:** We plan to generate a dataset of paired *parameters and resulting audio features* in a few ways: - **Re-amping DI through known chains:** We can take a set of DI guitar recordings (covering various riffs, tunings, pickups) and run them through a variety of amp/pedal settings. We could script popular amp sim plugins (or use open-source amp models like those in GuitarML/NeuralAmpModeler [14]) to create a large variety of tones. For each, we store the settings and analyze the output to get the feature vector. This yields a synthetic "training set" covering the mapping from tone settings to sound [15]. The Open-Amp project suggests leveraging crowdsourced amp models to create diverse training data [15] [14] – we can use a similar concept with the vast number of community-shared amp models to synthesize tones with many gear combinations.
- **Labeled tone packs:** We can leverage any existing tone packs or presets that are known (provided licensing allows). Many guitar communities share preset files for common songs. If those can be incorporated (again respecting copyright – likely we can use them for internal model training, but not redistribute verbatim if they're someone's IP; instead we might "learn" from them). At least they provide ground truth examples of what parameters yield what iconic sound. - **Manual curation for critical tones:** For MVP, we might hand-craft a small library of say 10-20 cornerstone tones (e.g., a baseline djent tone, a classic thrash tone, a death metal scooped tone, a black metal buzzy tone, etc.), capturing the wide range of heavy metal. This ensures we have at least one good example for each major style that retrieval can latch onto. - **Tagging and metadata:** For each library entry, we'll have tags like "artist:Metallica", "genre:thrash", "characteristics:scooped, mid-saturated, tight". This helps with text queries and also allows filtering (maybe user can filter "show me all modern metal core tones" etc. in a UI if they want).

We expect the library to provide a strong starting guess ~80% of the time, which then the next strategy can refine.

C. Model-Based Regression (Learning to Predict Settings):
This approach entails training a machine learning model that directly maps from **audio features → recommended parameters** (or from features of "desired tone" plus maybe features of user's guitar DI → parameters). Essentially, the model would try to *infer the knob settings* that would produce a tone with given features, learned from examples. For instance, given a target that has feature vector X, the model outputs that Gain=8, Treble=-2dB, choose Amp "5150", choose IR "MesaV30", etc.

We could frame this as a regression or multi-task learning problem: - Input to model: features of target tone (we could also include features of user's raw DI or current tone to let it know context). - Output: continuous values (for knobs) and categorical (for amp/cab choices, pedal on/off). - We might train separate sub-models: e.g. one model to classify which amp type fits best (based on the tone's spectral shape – e.g. if the tone has a certain midrange character, maybe it's likely a Mesa Mark vs a Marshall), and another to predict continuous knob values. - Model could be a simple neural network or even a decision tree ensemble. Neural net might handle the multi-output nature well. We'd likely use a fully connected network that takes the feature vector and outputs all parameters at once.

Training data: We need many examples of tone → parameters. The dataset we generate for retrieval can serve here too: we will have lots of random (or targeted) settings and we can compute their tone features. We can then train the model on those pairs. However, careful: randomly generated settings might produce a lot of "bad" tones (extremes that guitarists wouldn't actually dial in). We might want to focus training on plausible ranges (maybe restrict random sampling to reasonable ranges or bias around known good tones). Otherwise, the model might learn weird mappings that aren't actually optimal (like it might think extreme presence at max can produce a bright tone – true but also very noisy). It would be beneficial to include in training the curated good tones as well, so it knows what combinations are musically valid.

Pros: A model can give instantaneous predictions once trained – ideal for a real-time or interactive assistant. No iterative search needed; just feed features and get a result. It can also learn complex nonlinear interactions between parameters and tone that might be hard to express in rules. For instance, it might learn that if you want both high gain and clarity, you should lower Bass knob and add a boost – a combination of actions.

Cons: Generalization is a concern. The model might only reliably interpolate between tones it has seen. If a target tone is outside its training distribution (or the user's gear introduces differences), the prediction might be off. Also, training a model that includes discrete decisions (like amp type) can be tricky – typically you might either output a probability for each amp and pick the highest, or have a separate classifier. The accuracy needs to be high; even small errors (e.g. predicting gain 7 when needed 9) can make a difference in tone. We'd have to refine it possibly with human-in-the-loop or fine-tune on narrower styles. There's also the risk the model learns shortcuts or biases (like always recommending one particular amp if it dominates the training data, even when another would be better for a niche tone).

Given these factors, our plan is to **combine** these methods for a robust solution:

Hybrid Strategy (Recommended):
1. **Retrieve a candidate preset from the library** (if a good match exists). This uses either text tags or audio feature nearest-neighbor. This gives us a "starting point" TonePreset. If the match is very close to target features, we might skip heavy optimization. Usually though, it will be close but not perfect.
2. **Apply a refinement step**: Use **optimization** (like CMA-ES or Bayesian) initialized around that candidate to tweak the parameters for an even closer match. For example, treat the library preset's parameters as the center of an optimization search (with some allowed deviation range) to fine-tune EQ knobs or gain. Because the starting point is good, the optimizer can focus on a local region, likely converging in fewer iterations. If no decent library match is found (e.g. truly novel tone), we could fall back to either a heuristic initial guess (like choose an amp based on some spectral cues) or even default to a popular amp and run a broader optimization. But ideally the library covers most cases.
3. **(Optional) Use the regression model**: If we have a trained model by v1 or v2, we can incorporate its output as well. For instance, run the model to predict settings from the target features, and then use that as another candidate. We could even ensemble: generate suggestions from retrieval, from the model, maybe from a couple different retrieval entries (top 3 nearest), and then evaluate all of them with the objective function and pick the best or refine the best. This way the AI leverages both learned knowledge and direct search.
4. **Verification and final adjustments**: After deciding on a preset, we can do one final evaluation with the full feature set. If any of our *safety constraints* (as defined in evaluation) are tripped – e.g., the recommended tone is inadvertently much louder or has too much fizz – we automatically adjust or warn. For example, if output loudness is higher than input by > X dB, we reduce the output level in the preset until it matches (or

instruct the user to do so). Or if an extreme EQ curve came out, maybe flatten it a bit to avoid unnatural sound. This ensures the final recommendation is polished and safe.

Dataset Needs and Collection (for training & retrieval):
As mentioned, building a **comprehensive tone dataset** is a foundational task. Concretely, for MVP we might: - Use a **set of DI guitar recordings** that we either record ourselves or use from public domain/free sample packs (ensuring we have rights). It should include a variety: some chugs on low tunings, some power chords, some lead lines, different pickups if possible. - Choose a selection of amp models (software or neural models) common in metal: e.g., Peavey 5150, Mesa Dual Rectifier, Marshall JCM800, Engl Fireball, etc., and a few pedal + cab combos. - Randomly generate hundreds or thousands of presets: vary gain, EQ, presence, pedal on/off, etc. (We can constrain ranges to avoid totally crazy combos like zero distortion or something unless we also want clean examples for completeness). - Render these through the amp sim offline (could use an automated plugin host or an open-source simulation environment). Tools like OpenAmp or GuitarML's repository might help us automate generation [14]. Each run yields an audio which we then analyze for features. - Store the (parameter settings, features) pairs. This forms the basis for both training any ML and populating the retrieval feature database. - Additionally, incorporate **real-world presets** if possible: e.g., use Neural Amp Modeler community captures labeled with amp names (if those have any direct audio, maybe not, but if we can generate audio through them similarly). - Ensure to cover edge cases: e.g., extremely scooped EQ, extremely high gain, etc., so the model sees the full spectrum of possibilities and the retrieval DB has those corners.

Generalization and Limits:
No matter what, there will be limits: - **Different pickups & guitars:** As noted, if our dataset is generated mostly with one input DI (say a guitar with moderate output pickups), the model or retrieval might assume that tone behavior. If a user uses an 8-string with very hot pickups, the tone might saturate more or have different frequency response (more low end). Our system might mis-predict because the input itself is different. **Mitigation:** We should include a variety of input DIs in the dataset (we could simulate different pickups via EQing the DI perhaps, or actually use multiple guitars if available). Also, we could incorporate the **user's DI tone as part of the input** to the model – e.g., feed the model some features of the user's *dry guitar* (like its average spectrum or loudness) so it can adjust (this is an advanced idea; at minimum, we might ask the user to specify their pickup type as metadata). A calibration step could also be used: e.g., have the user play an open string and measure how loud it is and its frequency content to gauge pickup output and tone, then adjust gain/EQ expectations accordingly. - **Modeling error:** If we rely on a regression model, it may not always be accurate, especially on combinations it didn't see. But by combining with search and retrieval, we minimize catastrophic failures. For example, if the model suggests something bizarre, the objective check can catch that it doesn't actually match well and then an optimization or alternate suggestion can override it. - **Complex multi-amp tones:** Some iconic tones are actually a blend of two amps or have unique studio processing (e.g., one amp for lows, another for highs). Our chain currently assumes one amp/cab. We won't capture the nuance of dual-amp blends in MVP (that's an assumption). Thus any tone that was originally a blend might be approximated by one chain. This might be a limitation for, say, certain Metallica or Tool tones where layering is key. We will have to live with that and perhaps note to users that "the result is a single-chain approximation of the layered tone". Possibly in future we consider multi-amp chain suggestions, but that doubles complexity. - **Extreme effects outside scope:** If a tone has an outboard effect (like a flanger or pitch shifter used artistically), our system isn't handling those – we focus strictly on the core amp/distortion tone. Users should understand that (we could disclaim that the assistant won't add special effects like delay, reverb, modulation, which are not part of "core tone" here). - **Real-time adaptation**: We expect the offline suggestion to be the primary. If doing live assist, the strategy

might be simplified (likely not running full optimization constantly, but maybe using the trained model to make quick minor adjustments in response to live feature tracking). Generalization for live use means the model must be robust in streaming mode; that's later version consideration.

In conclusion, the recommended solution is **Hybrid**: retrieval to **get close quickly**, plus optimization to **fine-tune precisely** [11], possibly augmented by a predictive model for speed. This leverages existing tone knowledge (library/dataset) and algorithmic rigor for accuracy. We will invest in creating the necessary dataset (both synthetic reamps [14] and curated presets) to support this. The result should be a system that usually finds a great tone match within seconds and handles a wide range of inputs. Next, we discuss how this all fits into the overall product architecture and how it will be delivered to the user.

6. Integration Architecture (Product & Engineering)

We outline an architecture that allows the Tone Assistant to function as part of a desktop application or plugin, balancing heavy offline analysis with the responsiveness needed for user interaction. Key modes are **Offline "Analyse & Suggest"** and **Optional "Live Assist"**, and we need to integrate with plugin software for applying settings and exporting presets. Below is an overview of the system components and their interactions (as a conceptual diagram in text form):

User Interface (Frontend):
- This could be either a **standalone desktop app GUI** or a **plugin UI (VST/AU)** that runs inside a DAW. Given the constraint "desktop for now" and focusing on VST integration, a plausible approach is to implement the assistant as a **VST3/AU plugin** that can be inserted on a track. This plugin would have a UI panel for the Tone Assistant. It might internally host the actual amp sim plugin or simply control it externally (two integration styles discussed below). Alternatively, a standalone app could be used where the user loads audio and selects target, then exports a preset file. For maximum usability, embedding as a plugin is better (the user can use it in their usual workflow, on the same track as their guitar).
- The UI displays controls for the assistant: - Input controls: e.g. a button to **load a reference audio file**, or record a snippet of the user playing, or a text box to type a tone description. - A **"Suggest Tone"** button to initiate analysis & recommendation. - Options for constraints (checkboxes like "Use boost pedal", "Prefer my existing amp sim"). - Output controls: showing the recommended chain (could be a list of blocks with knob positions). Possibly interactive (user can tweak knobs right there). - **A/B toggle**: allow user to hear before/after easily. If our plugin is hosting the amp sim, it can bypass itself or revert settings for A/B. If it's external, maybe it loads the preset into the user's plugin but allows toggling to previous settings. - If live assist mode is on, perhaps a toggle for "Live Mode" which continuously monitors input and suggests adjustments on the fly (or just displays hints like "Try lowering gain" in real time). - **Status display**: show progress of analysis (since some seconds may be needed), and results like similarity score or what was matched ("Matched to: Mesa Dual Rec + TS9 + Oversize V30 cab"). Also warnings if any (e.g. "Reference contained non-guitar audio – results may be approximate").

  - The UI communicates with the backend logic either via direct function calls (if integrated) or via an IPC if separated process. It remains responsive (maybe multi-threaded or using async tasks) so that the DAW doesn't freeze during analysis.

Backend Analysis Engine:
- This is the core logic that performs **audio feature extraction** and **tone matching computation**. It can be implemented in a high-level language like Python (for ease of using scientific libraries) or C++ (for speed).

There are two patterns: 1. **In-process (C++ or JUCE based)**: Implement analysis and optimization in C++ inside the plugin. This could use libraries like JUCE DSP, Eigen for math, etc. It avoids cross-process communication and can leverage real-time audio stream if needed. But writing complex optimization in C++ might slow development, and using Python ML models would require embedding a Python interpreter or converting models to C++ (e.g. via ONNX). 2. **Out-of-process service (Python or microservice)**: The plugin UI/frontend calls a separate local service (could be a Python server or even a small local server) that does the heavy computation. For instance, when the user clicks "Suggest", the plugin collects the relevant audio (reference and maybe a DI sample) and sends it to this backend service. The service (running Python with e.g. librosa, PyTorch, etc.) performs feature extraction, runs the retrieval and optimization, and returns the resulting TonePreset. The plugin then applies it. This separation can make development easier and allow using powerful ML frameworks in the backend without bloating the plugin. The downside is complexity of packaging two components and ensuring low-latency communication. However, since the main heavy work is offline (not continuous streaming), a local HTTP or socket call with a few seconds response is acceptable.

Given "desktop for now" and likely emphasis on reliability, an **in-process approach** using C++ (with possibly some pretrained models converted to C++) might be preferable for a polished product. We might prototype using Python, but eventually port critical parts to C++ for integration. We will aim to design with modular boundaries so that replacing a Python prototype with a C++ implementation is straightforward.

Audio Routing and Processing:
- In **offline analysis** (matching a reference): we need to analyze the reference audio and possibly the user's playing through candidate settings. If the assistant is a plugin inserted on a track, we have access to the user's **DI signal** (assuming they put it on the DI track pre-amp sim). Actually, if our assistant plugin *hosts* the amp sim, then it naturally processes the DI through the chain. If not, we may need the user to route their DI to us for analysis. Possibly the user could record a short DI snippet and press "Analyze" – our plugin grabs that snippet. Alternatively, if integrated as a standalone app, the user would import a DI file. For reference audio, the user will likely import an audio file (WAV/MP3). The assistant needs to read that (we'll include audio decoding capabilities). - For the **tone matching algorithm** (especially if doing optimization), we have to **apply various settings to the amp chain and get resulting audio** to compare to reference. There are two ways: 1. **Host the DSP chain within our system:** Essentially, our assistant can act as a lightweight DAW host itself – loading the amp sim plugin and any needed pedal plugin inside it (like how biasing some host that can load VSTs). JUCE or similar frameworks allow loading a VST plugin in code. If we do that, our code can programmatically set plugin parameters, render a buffer of audio through the plugin, and measure the output. This is ideal for automation. For example, during optimization, to evaluate a candidate parameter set, we would set those on the amp sim plugin instance, process the user's DI through it (maybe a short segment or the whole thing), and compute features. This requires the amp sim plugin to be available on the user's system and loadable (which in this scenario it is, since they own it). We need to ensure licensing or headless usage is okay (most plugins allow being loaded by any host). The complexity is moderate – essentially writing a mini audio engine, but since we already are a plugin, we can possibly leverage the DAW's processing if we sequence it carefully. More likely, we will spawn our own processing since we might want to bypass the DAW's timeline. 2. **Control the user's existing plugin instance:** If the assistant is not hosting the amp but just "observing" what the user's existing chain is, it could attempt to manipulate that chain's plugin parameters via the DAW's API (e.g., some DAWs allow scripting or if the plugin and host support MIDI/OSC control of parameters). However, this approach is **highly DAW-dependent and not reliable** across systems. Also, reading the plugin's state to know what amps/cabs are loaded might be impossible externally. So this is not ideal for an automated tool.

Given the above, it's safer to have the assistant **host at least the amp and pedal plugins inside itself** for the analysis stage. One model: The assistant plugin could itself be structured as a *wrapper*: you insert it on a track, and inside its UI, you select which amp sim plugin you want it to control (maybe a dropdown of detected installed amp plugins or profiles). Once selected, it loads that plugin internally (in a disabled/bypassed state initially), and can manipulate it. The user's DI flows through our plugin into the internal amp plugin, then out. This way, our plugin essentially replaces the direct usage of the amp sim – the user still gets the audio output of the amp sim through us, but we have control hooks. This is similar to how iZotope's Tone Match in Neutron plugin might host or control EQs. It's complex but feasible with proper plugin hosting code. For MVP, we might restrict support to a particular plugin chain (e.g., maybe we build our own simple amp sim as part of the assistant to demonstrate the concept, or use one known VST that's easy to control).

Interoperability and Preset Application:
- **If hosting internally:** after computing the TonePreset, we directly set the parameters on the internal plugin(s) to those values. The user hears the result immediately. We also can export it: either via the plugin's own preset save (if accessible) or by writing out the preset file format (discussed in section 7). - **If standalone app approach:** after computing, we generate a preset file or list of settings for the user to manually apply in their amp sim. Standalone has the disadvantage that the user then has to load that preset in their DAW to actually hear it – not as seamless, but possibly acceptable for an offline analysis tool.

Service Boundaries & Data Flow:
- **Audio Analysis Worker:** Whether in-process or out-of-process, treat the heavy number crunching as a separate module. It takes inputs (audio buffers from DI, reference audio file, perhaps initial guess preset) and runs feature extraction + algorithm, returns a TonePreset. We must ensure thread safety (if in plugin) or proper async handling (if out-of-process, likely we'll call it asynchronously and get a callback or poll for result). - **Model Inference (if any):** If we have a ML model (like a neural net for parameter prediction or a learned similarity metric), that is part of the analysis worker. If Python-backed, it could load a PyTorch or TensorFlow model. If we port to C++, maybe use ONNX runtime or a lightweight inference engine. The architecture should allow swapping that model out (like new model versions as we improve). - **Preset Library Storage:** We will have a local database or file that stores the preset library (for retrieval). This could be a JSON or a small SQLite database containing presets and their feature vectors, plus metadata. On initialization, the assistant loads this library (or at least indexes it). We might allow updating it (maybe downloading new tones from our server if this is a product with online updates, but given offline focus, it could ship with a static library file). - **Caching & Determinism:** If a user repeats the same analysis (same reference, same input conditions), we want the same result each time (assuming same version of the algorithm). This means the process should be deterministic. Evolutionary algorithms have randomness, but we can set a fixed random seed per analysis to ensure repeatability. That way, if the user is A/B testing our assistant's suggestion with a mix, they won't get a different suggestion each time they click (which would be confusing). We also can **cache results**: e.g., if the user matched "Song X" before, store the resulting preset in a cache (maybe keyed by an audio fingerprint of the reference). Next time, just retrieve it instantly rather than recompute. This cache could be in memory or disk (for persistence across sessions if desired). But we should also allow a "recompute" if the user changed guitars or updated the algorithm. Maybe include in the cache key some fingerprint of the user's DI tone or current chain, and definitely the algorithm version. If we release an update with improved matching, we might invalidate or version the cache so old results aren't blindly reused.
- **Feature Flags & Rollout:** Early versions of this assistant might be marked as beta. We could include a hidden setting to toggle certain experimental parts (for example, using the ML model vs purely

retrieval+opt, or enabling live assist). These can be controlled with feature flags, which in a standalone app could just be config settings or in a plugin possibly special key combos to enable a mode. For rollout, if this was a commercial product, one might do a staged release (internal testing, closed beta, then general). The architecture doesn't necessarily need explicit support for rollout beyond maybe an update check. But since mentioned, we ensure that any potentially risky features (like live mode) are off by default until proven stable.

Live Assist Mode Architecture:
In live mode, the assistant is continuously analyzing the incoming guitar in small chunks (e.g. every second or every few bars) and could make small adjustments on the fly. This mode must be very light and not disrupt audio (no long computations). Likely the heavy stuff (like library search or large optimization) won't run live. Instead, live mode could rely on the trained model or simpler heuristics. For example, a live algorithm might monitor if the user's tone has drifted (maybe their volume changed or they switched pickup) and then adjust gate threshold or gain slightly to maintain consistency. Or it might simply flash suggestions on screen ("Increase mids for clarity") rather than actually change knobs live, leaving it to the user to tweak in real-time if they choose (because unexpected auto-changes while playing could be distracting). Implementation-wise, if the assistant plugin is hosting the amp, it can adjust the parameters in real-time – but we'll be cautious not to do anything abrupt that could create noise (smoothing any changes). This is an advanced feature for v2; initially, we focus on the offline suggestion which has the above robust pipeline.

Interoperability and Plugin Communication:
The architecture also needs to support exporting presets to third-party software. We cover that in detail in the next section, but from an architecture view: we might have a **Preset Export Component** (a module that knows how to format and write preset files for various plugins). The UI could have an "Export to <plugin> preset" button which triggers this component with the current TonePreset and target plugin selection. If the assistant is itself hosting the plugin, it might also directly call that plugin's save function if available. But many plugins don't expose a save, so writing to file is safer.

Additionally, if our assistant is used as a standalone app, it might need to **interface with DAW projects** in a limited sense – e.g., reading the user's current plugin settings to use as a starting point for refinement. That would be tricky, but maybe an alternative usage: user exports their current preset from the amp sim, loads it into assistant as "current tone", and then the assistant knows starting parameters. Possibly out of scope for now, but something to consider (ease of import of user's current state so we know what to improve).

Logging and Monitoring:
During development and testing, we'll have verbose logging (to files or console) in the analysis engine to trace decisions. For production, this can be toned down or behind a debug flag. If any errors occur (e.g. plugin could not load, or analysis failed), the system should catch and inform the user gracefully ("Sorry, unable to load the selected amp sim" etc.).

Security and Privacy:
All processing is local (no cloud), so user's audio stays on device. If we had an online component (like pulling new presets from a cloud library), we'd ensure it's optional and transparent. For now, everything is self-contained.

Summary (Architecture Diagram Description):
- **Frontend UI (Plugin)** –> triggers –> **Analysis Engine** (feature extraction, retrieval, optimization/model) –> yields –> **TonePreset** –> passed to –> **Preset Mapper** (maps TonePreset to specific plugin parameters or file) –> outputs –> **applied settings** (in hosted plugin or exported file).
- The **Tone Library DB** and **ML Model** are accessed by the Analysis Engine to assist in suggestions.
- The **Audio path**: User's DI enters plugin -> either goes straight to output (bypassed) when not engaged, or through the chain when we apply suggestions. During analysis, the engine may internally route the DI through various virtual settings. Reference audio goes only into analysis engine (not to output).
- **Data stores**: Preset cache stores previous suggestions keyed by fingerprint. Preset library stored on disk. Possibly user presets store (if they save suggestions to a personal library, that could be a new entry in library DB too).

This architecture ensures modularity (we can update the analysis algorithms or library without touching UI), and real-time safe operation (heavy lifting in either a separate thread or process). Now, having described this integration, we proceed to the specifics of preset export and format compatibility with third-party amp sim software.

7. Preset Export: Concrete Formats and Strategy

A key requirement is that the Tone Assistant can **export presets for popular software amp sims**, so that users can readily use the suggestions in their preferred plugins. We consider several strategies for preset export and decide on a combination:

Option A: Native Preset Format Export – The assistant directly generates a preset file in the exact format used by the target plugin (for each supported plugin). The user could then import this file into their plugin (or if our assistant is hosting the plugin, possibly load it automatically).

Option B: Intermediate Generic Format + Converter – The assistant could output a vendor-agnostic JSON (essentially our TonePreset schema instance) and provide small scripts or tools to convert that JSON to specific plugin presets. This adds an extra step for the user unless integrated, but might simplify our code by focusing on one format (JSON) and maintaining separate converters.

Option C: Real-time Parameter Tweak / MIDI Automation – The assistant could output a sequence of parameter changes (e.g. a MIDI CC or automation script) to apply to a plugin instance. For example, some amp sims allow controlling knobs via MIDI CC assignments; our tool could generate a MIDI file or set of CC messages that, when played, set the knobs appropriately. Or if integrated as plugin, it could just set them directly. This is a more niche solution and not as user-friendly as just giving a preset file.

We will prioritize **Option A (Native Preset Export)** for immediate user convenience. Where needed, we'll supplement with Option B (for plugins whose format is too complex or if we choose to allow advanced users to see the JSON). Option C might be used internally if we need to apply settings in a live host scenario, but not as a primary export method.

Supported Targets (plugins/software) & Rationale:
We choose a set of popular amp sim platforms among metal guitarists: - **Neural DSP plugins** (e.g., Archetype: Nolly, Gojira, Plini, Fortin Cali, etc.): These are widely used for high-quality tones. Their preset

format is typically an XML or JSON inside a `.preset` file (for example, Neural DSP presets are often in XML with parameters). We support them because many modern metal tones are achievable there, and these plugins have consistent structures (amp + cab + pedal sections). - **Positive Grid BIAS FX / BIAS Amp:** Given BIAS X is explicitly an AI tone product, supporting BIAS might seem redundant. However, some users might want to use our independent assistant with BIAS as the engine. BIAS FX presets might be in JSON (Bias Amp used to have text JSON-like format for custom amps). We need to investigate format or use their ToneCloud API if accessible. Possibly de-emphasize if complicated, but include if possible. - **Line 6 Helix Native (and HX Stomp)**: Line 6's Helix family is popular for metal (and Helix Native plugin uses the same preset format as hardware Helix, which is a JSON structure with extension `.hlx`). It includes amps, cabs, etc. We can generate a Helix preset file by inserting our suggested blocks into their JSON structure. E.g., choose a certain amp block code that matches our recommended amp (Helix has codes for each amp model), and set its parameters. Same for cab (if using stock cab, pick closest, or Helix also allows loading third-party IRs referenced by a filename). - **IK Multimedia AmpliTube**: AmpliTube is another common suite. It has a preset format (possibly `.atxp` or similar XML). If it's accessible, we'll support it, focusing on the amps and cabs we care about. - **Native Instruments Guitar Rig or STL ToneHub, etc.:** These are also in market. Guitar Rig less metal-focused; ToneHub (by STL Tones) is essentially preset packs from producers – might not allow external preset creation easily. Maybe skip in MVP. - **Fractal Axe-Fx**: mostly hardware, though Axe-Edit uses .syx files. Not a plugin (except their newer FM9 plugin maybe not available). Probably out of scope due to hardware and closed format. - **Others**: Possibly support **Mercuriall** (some specialized amp sims), **Kazrog** Thermionik, or open source ones. But focusing on big names covers majority.

We will pick maybe the **top 3-4** to implement first: **Neural DSP**, **Helix**, **BIAS**, **AmpliTube**. The rationale: - Neural DSP: very high quality, many metal artists use it, likely our prime recommendation environment. - Helix Native: versatile and many have it, plus it has a breadth of amp models that cover many tones. - BIAS FX: since it's explicitly marketed as AI tone, but we can offer an alternative or complement. Also, they have an accessible system perhaps. - AmpliTube: popular and many amps licensed there. If time, possibly add Overloud TH-U (used for Randall, Mesa, etc) or others.

For each supported target, we'll create a **mapping table** from our TonePreset schema to that format: - **Amp Model Mapping:** We will map our abstract amp names to the specific model name or ID in the plugin. E.g., TonePreset says `amp.model = "Peavey 5150"`. In Helix, the corresponding model might be "Cali IV Lead" (just an example name if Helix has a 5150 model) or perhaps "PV Panama" (Helix code name for 5150). We maintain a dictionary for each target plugin: e.g., `{"Peavey 5150": {"helix": "Cali IV", "neural_nolly": "Lead 33", "amplitube": "5150III model name", ...}}`. If an exact amp isn't present in a given plugin, we choose the closest alternative (based on topology and sound). For instance, if user target is a Peavey 5150 but the user only has a Marshall in their plugin, we might just pick a high-gain Marshall and try to compensate via EQ (but we should indicate that it's an approximation). - **Pedal Mapping:** Similarly, our `pedal.model = "TubeScreamer"` maps to, say, Helix's "Scream 808" block, Neural DSP might have a built-in overdrive (Archetype Nolly has an OD pedal in it, which might not be exactly TS9 but similar). We either toggle that on and set tone/level equivalent, or if plugin has no boost pedal, we might emulate it by adjusting input EQ/gain. - **Cab/IR Mapping:** This is a bit tricky: - If the target plugin allows third-party IR import (Neural DSP and Helix both do), the simplest route is to have the assistant also export the IR file (if it's a known free IR we have) or instruct the user to load a specific IR. We could package some free IRs with our assistant (ensuring they're license-free). For example, if our suggestion is "Mesa V30 with SM57", we include a free IR with that spec (there are many free IRs of Mesa cabs). Then for export, if the user chooses Helix, we can either embed an IR or provide the IR file and set the preset's IR slot to point to it (Helix preset references user IR slots by index, not sure if Helix Native can

automatically load a WAV IR via preset – maybe not, user might have to import IR into slot 10, then preset refers to slot 10). Possibly we just supply the IR and instruct the user to import it into their plugin's IR loader. - If the plugin has a built-in cab model and the user prefers that (maybe they don't use IR files), we map to the closest built-in cab. e.g., in AmpliTube, if Mesa V30 4x12 with SM57 is available as a cab option, we choose it. It won't be identical to a specific IR, but it's consistent with that environment. - We will include metadata like mic and position: in Helix, for example, if using stock cabs, you can choose mic type (SM57, etc.) and position (distance). We'll set those to match our target metadata as best as possible. In Neural plugins, often the cab section allows choosing different IRs or mics; if accessible, we do similarly.

- **Parameter scaling:** Once we choose the block models (amp, pedal, cab), we set parameters. The mapping needs to account for differences in scale and default:
- E.g., our TonePreset might say amp Bass = 0.4 (40% of knob). In Neural DSP, the Bass knob might range 0-10; we'd set it to 4.0 if linear. In Helix, Bass might be in dB or an arbitrary scale, but likely 0-10 as well. Some differences: e.g., Helix's presence might be a dB value (like -5 to +5 dB) rather than 0-10; we'd convert our normalized presence to that range (maybe 0.7 normalized = +2 dB, for instance, after calibration).
- If an amp has no Depth knob but our preset had one, we ignore it or approximate by an EQ if needed. If an amp has an extra control we didn't consider (say "sag" or "bias"), we leave it at default (the preset or plugin will have its default that presumably was used in library).
- Ensuring no conflicting values: e.g., if the plugin preset file requires values for all parameters, we must fill them all. We can fill unspecified ones with some known defaults or those from the library base preset.

Known Incompatibilities & Checks:
- **Differences in Ranges/Linearity:** We know that e.g. "Gain 7" on one amp sim might not equal "Gain 7" on another in actual distortion amount. Our approach tries to mitigate via the matching process (we match by sound, not by knob value). So, if we produce a TonePreset for an abstract 5150 with gain X, that value X was determined to get the sound on presumably a model of 5150. If we then map X to a different 5150 sim, it might not be exact. For safety, after exporting, if possible we could **validate by re-running our feature extraction on the exported preset's sound** (this requires loading the preset into the plugin either automatically or instructing user to do a test). In practice, this might be too heavy to do for every export. Instead, we rely on having used the same plugin for matching as the user intends to use. This hints that perhaps during analysis, we should know which target plugin the user will use, so we can use that plugin's model if available for the best accuracy. (We might ask user "Which plugin do you want this preset for?" at start. If they choose Helix, we would try to use Helix's amp models in the loop rather than, say, using Neural's and then converting, to avoid differences.) - **Proprietary or Encrypted Formats:** Some preset formats might be not easily hand-editable (maybe encrypted or binary). For instance, BIAS presets or ToneCloud might not be open text. If so, we have options: use their API if any, or reverse-engineer format if legal, or use a workaround (like instructing user manually). We must check license terms; generating a preset likely is fine as it's user's right to create presets if they own the software. If it's too closed, we might deprioritize that platform or use intermediate instructive output (like "Set amp to Insane 5150, gain X, etc."). - **IR management:** If we export to a format that doesn't embed IR files (like Helix doesn't embed the wav, it references a slot), we ensure to provide the IR to the user. Possibly we zip the preset and IR together or output a folder. In any case, documentation or on-screen instructions will clarify any manual steps ("Import the included IR into slot 3 of your plugin, then load the preset"). - **Round-trip validation:** We can implement a developer-mode check where after writing a preset file, we try to load it (if we have the plugin available headless) and render a short sample, compare to what we expected. This can catch if, say, we mis-ordered JSON fields or scaled a parameter incorrectly. If the loaded tone is off, we adjust our mapping. For shipping,

likely this step would be done in testing rather than each time for user (except possibly a quick check on safe values).

Export UI/Workflow:
- The UI might have an “Export Preset” dropdown: the user picks the target software (if not already specified earlier). Then clicks export, chooses a file name/location. We generate the file with the correct extension (e.g., `.nollypreset`, `.hlx`, `.preset`, `.fxp` etc as needed). - We should show a confirmation like “Preset for <PluginName> saved successfully. Load this in <PluginName> to use the tone.” Possibly even offer to open it if that plugin is also our host (though if it’s a separate app, not possible). - If the assistant is hosting the plugin internally and the user is satisfied, they might not need to export – but it’s there if they want to take that preset to use standalone or share.

Intermediate JSON Option:
Even if we do native export, it might be useful to also allow exporting our internal TonePreset as a JSON. This could be for transparency or for use with a provided conversion script. For example, if the user’s plugin is unsupported, they could at least get the generic settings and try to dial them in manually or adapt them. This JSON could also serve as a log or for support (“send us the TonePreset JSON if something went wrong”). So we will include an option to save the generic preset as well. The generic preset plus a small script can handle conversion to any new format we add later, making extension easier.

Summaries for Each Supported Target (initially):
- *Neural DSP:* Presets are usually in XML form inside a file (with extension like .preset or .xml depending on product). We will reverse engineer one or two by saving them and inspecting. For example, a preset might list `<Pedal1 type="TubeScreamer" drive="20" tone="60" level="80" enabled="true">` etc., and `<Amp type="Soldano SLO"> ...`. We’ll create our preset file accordingly. Parameter mapping: linear 0-10 likely for tone knobs. Incompatibility: Neural DSP often includes fixed pedal/amp per plugin (e.g. Archetype Gojira has specific amps). Our suggestion might be to use a different amp model not in that plugin – but if user doesn’t own it, that’s moot. So likely if user says they want to export to Neural DSP Gojira, we confine suggestions to that plugin’s models (maybe the user picks which plugin they want us to use for results). - *Helix Native:* The .hlx is JSON with blocks. We identify block types for Amp, Cab, etc. Parameter ranges: Helix uses numeric values often corresponding to real unit values (e.g. “Drive”: 7.3, “Bass”: 5.0, etc. out of 10). We fill those. We have to mind the firmware version field in JSON, etc., but those we can get from an example preset. Incompatibility: If our chain includes a pedal Helix doesn’t have (though Helix has a TS and others, so fine). If it includes post-EQ, Helix has EQ blocks we can use (graphic or parametric), we can drop one in and set a band cut if needed. Good thing Helix allows multiple blocks, so we can pretty much build our entire chain in one preset (Gate, Distortion, Amp, Cab, EQ). - *BIAS FX:* Possibly uses a proprietary binary or stored in ToneCloud. If direct file is an issue, we might output an intermediate or use their software’s ability to import tone match via ToneCloud (maybe beyond scope). If needed, skip for MVP if format not easily accessible. - *AmpliTube:* We’ll research the format. If too closed, maybe skip or output instructions (like which amp/cab to use and dial numbers). - *In general*, as we implement each, we will test by exporting a preset, loading it in the plugin, and verifying sound or at least that the knobs match expected values.

We will also include a **validation method** for preset export:
- At least verify that the file written is parseable by the target’s software (perhaps by a schema or by trying to import ourselves if possible). - If possible, test that critical values made it (like the amp model name is

correct, etc.). - Possibly provide the user with a short summary: “Exported preset uses Amp X, Cab Y, with Gain=, Bass=, etc.” so they can sanity-check when they open it.

In conclusion, we will implement direct preset generation for key plugins, ensuring that for each one we have a robust mapping of our canonical settings to their specific parameters. This will let users seamlessly transition from the AI suggestion to actually playing through that tone in their environment. Next, we address how we will evaluate the success of the Tone Assistant and keep it safe and reliable.

8. Evaluation Plan

To ensure the AI Tone Assistant works as intended, we develop a comprehensive evaluation plan covering offline metrics, user listening tests, regression tests for edge cases, and safety validations.

Offline Evaluation (Algorithmic):
We will assemble an **evaluation dataset** that includes: - A set of **reference guitar tones** along with known “ground truth” settings or at least known target characteristics. For example, we might take a few *commercial isolated guitar tracks* from well-known songs (if available through stem packs or re-amped covers) to use as references. Or we create references by re-amping DI through a known preset, so we actually know the correct answer. - Corresponding **DI inputs** for each reference (the same riff played clean). If we have the reference as re-amped output, we presumably have the DI used to create it (especially if we make them ourselves). This allows a realistic test: feed the DI to the assistant and see if it recovers the original tone settings. - Include diversity: high-output humbucker vs low-output single coil DI, 6-string in E, 7-string in drop A, etc. – to test how the system handles different input spectra. - Include different styles within metal: from classic thrash (which might be lower gain, more raw) to ultra modern djent (super tight, gated), to sludge/doom (fuzzier, lower mid heavy), etc. This ensures the assistant isn’t overfitting to just one type of “metal tone”.

For each evaluation item, we will run the Tone Assistant in automated fashion (with no user tweaks) and then measure: - **Feature similarity scores** between the output tone and the target reference. Using the same metrics as in our objective function: e.g., compute spectral error, etc. Ideally, these should be below a certain threshold. We’ll define acceptance criteria like “90% of evaluation cases have spectral mismatch < 3dB in all critical bands, and transient match within X%”. - If we have ground truth parameters (because we created the reference with known plugin settings), we can also measure **parameter error**: e.g., did it pick the correct amp model? How far off are the knob values? For continuous knobs, a small difference might still be fine if the resulting sound is close (due to the many-to-one mapping sometimes). But this can catch if, say, it completely missed that a boost pedal was needed. - We also evaluate the **time taken** for each suggestion – to ensure it’s within acceptable limits (if one case took 2 minutes, that’s too slow; we aim for maybe <30 seconds worst-case on a mid-range PC for offline match). - We’ll run these tests whenever we refine the algorithm (regression tests) to see if changes improve the scores or if any scenario worsened.

Human Listening Tests:
Automated metrics are good, but ultimately tone is subjective. We will conduct listening tests with a panel of guitarists or audio engineers: - **ABX Tests:** For a given reference tone and the assistant’s matched output, do listeners perceive them as the same or different? ABX means they hear A (reference), B (assistant output), and X (randomly either A or B) and must guess if X is A or B. If our matching is successful, listeners will often fail to distinguish (50% guess rate). We’ll do this for several tone scenarios to quantify perceptual match. - **Preference Tests:** If exact match is not the goal for some (like text-based suggestions), we can

compare our suggested tone (for say “djent tone” on a user’s DI) vs the user’s original attempt or vs a generic tone. Do players prefer the assistant’s result? Ideally yes, it should be rated higher for descriptors like “closer to what I described” or “more mix-ready”. - **Attribute Rating:** Have experts rate the output vs reference on specific qualities: - Tightness of low end (e.g., scale 1-5 how tight). - Clarity of notes. - Fizz/harshness level. - Authenticity (if matching a known artist tone, how close does it feel to that artist’s known tone). - Overall quality or readiness to use in a mix.

We’ll gather feedback and look for patterns (e.g., maybe it matches EQ well but testers say “feels more compressed than original” – that tells us to improve dynamic matching).

Continuous Testing (CI):
We will integrate automated tests into our development cycle: - **Unit tests** for feature extraction (e.g., feed a known test signal and verify the features like centroid, etc., are computed correctly and consistently). - **Integration tests** for the optimization loop (e.g., test on a simple known scenario: maybe a pure EQ match case to see if it finds the right EQ). - **Regression tests**: Over time, as we fix bugs (especially in edge conditions), we add test cases to ensure they don’t recur. For example, if we find an issue where the assistant tended to set gain too high (“over-gain”), we create a test to detect that condition and ensure future changes fix it and don’t revert.

Edge-case and Adversarial Testing:
We specifically check for the problematic outputs enumerated: - **Over-gain**: We simulate a scenario where reference is moderately distorted but maybe our algorithm overshoots gain. We measure dynamic range or THD. We can create a threshold (like if output crest factor < some small value, it’s probably too compressed beyond reference). If our suggestion triggers that, it fails the test. The fix might involve clamping gain or using the noise floor features. - **“Ice pick” highs**: We test if any output has a strong peak in 3-5 kHz region > say +6 dB relative to rest of spectrum. That would be flagged as likely unpleasant. We’ll adjust algorithms to avoid that (perhaps by adding an EQ cut automatically if needed). - **Mud/boom**: If any output’s <100Hz content is disproportionately high (like sometimes too much resonance around 120Hz from a cab can muddy a mix), we flag it. The assistant could then incorporate an automatic slight high-pass or a note to user. In tests, we can quantify by low-frequency energy ratio threshold. - **Thinness**: Conversely, if our output has significantly less low-end than reference (maybe overshot trying to tighten), that’s a mismatch. We ensure the low band energy ratio is within some tolerance of reference’s. - **Noise and stability**: We will test that with no input signal (silence), the assistant’s output chain is not adding a lot of hiss or hum. This means the noise gate should be doing its job. We can automate: feed silence into the suggested preset and measure output level; it should be near digital black. If not, maybe the gate threshold is too low or off – adjust accordingly. Similarly, test if extremely high input (simulate a very loud pickup) causes any internal clipping beyond what amp sim would normally do; ensure output doesn’t go beyond 0 dBFS or whatever safe level we define (we might incorporate a soft limiter at end just in case). - **Algorithm stability**: Minor variations in input should not cause wildly different suggestions. We can test by adding slight random noise to the reference or DI and see if the outcome changes drastically. It shouldn’t – if it does, maybe the optimization is latching onto noise; we might need more smoothing in features.

Safety Checks:
Hearing safety is critical: - We make sure the assistant never introduces unexpected volume boosts. The final output preset should be loudness-normalized to the reference or a standard level. In evaluation, we will explicitly measure the LUFS of the output vs input. A check: if the output preset’s tone is more than e.g. 1-2 dB louder in perceived volume than the reference tone (or the user’s original tone), we will reduce the

level parameter or output knob. Many amp sims have an output volume; we can tune that. Or if not, add a suggested gain adjustment. The user's ear should not be shocked when toggling A/B. Ideally it should be level matched so that "better" tone is not just "louder". - Check for any potential feedback or oscillation: Typically in software amp sims, feedback isn't an issue unless input is looped. But if our assistant, say, set gain to max and no gate, and user leaves strings open, maybe squeal. However, since it's just sim, it won't feedback without physical loop. So less of an issue than in real amps, but heavy noise could still annoy. We mitigate by gating. - If live mode adjusts parameters, ensure no rapid oscillation or extreme jumps that could cause pops (like don't jerk the volume knob quickly). We can simulate quick changes and listen or check for discontinuities (maybe more of a QA manual test for audio smoothness). - Ensure any **suggested IRs or processing** do not contain dangerous frequencies (no DC offset, no ultra-sonic noise). IRs might have a lot of high-frequency content; we can apply a gentle roll-off above, say, 12-15 kHz in our chain because guitars rarely need beyond that and it avoids any potential aliasing in some plugins.

Human Beta Testing:
Finally, before release, recruit a handful of target users to try the assistant on their own setups. They can give qualitative feedback: - Did the suggestions make their tone better? - Were any suggestions clearly off or "weird sounding"? - Was the workflow intuitive (or did they find it confusing/hard to trust)? - Any crashes or performance issues on their system? - They might also test unsupported scenarios (like weird input signals) that we didn't think of.

Use this to fine-tune defaults and fix any issues.

Continuous improvement metrics:
We can instrument the assistant (if privacy policy allows) to collect anonymous usage stats: e.g., measure how often users accept a suggestion vs tweak further, or how often they click "retry" or adjust the descriptive prompt after hearing the result. This can hint at if the first suggestions are usually satisfactory. If many users always have to tweak treble down after using it, maybe we know we overshoot treble in suggestions. This is more long-term but helps v2 refinements.

In summary, our evaluation plan is multi-faceted: automated objective measures ensure technical correctness, listening tests ensure perceptual quality and user satisfaction, and targeted regression tests ensure we avoid known pitfalls like fizz, mud, or dangerous levels. This will help us iterate to a robust solution and catch issues early. Now we consider potential risks in more detail and how we mitigate them.

9. Risks and Mitigations

Designing an AI tone assistant involves several risks, both technical and user-experience-wise. Here we enumerate major risks and how we plan to address them:

- **Risk: Reference clip contains drums/bass or other mix elements, confusing the tone analysis.**
  Mitigation: We will incorporate a **reference content check**. Before trusting the extracted features, analyze the reference spectrum for signs of non-guitar content. For example, if there's significant energy below 50 Hz or sharp transient patterns inconsistent with guitar strums (likely drums), flag it. We might attempt a **basic source separation**: using a pre-trained model to isolate guitar (there are some open models for music source separation). If the separation works, use the isolated guitar for analysis. If not, we fall back to either asking the user for a better reference or focusing only on frequency regions dominated by guitar (e.g., 100Hz–5kHz for guitar and ignoring sub-bass thumps

and cymbal highs). We will also communicate to the user: if a mixed track is provided, show a warning like "The reference seems to contain other instruments; results may be less accurate. For best results, use an isolated guitar track." This sets expectations. In testing, we'll ensure that if a full mix is given, the assistant doesn't naively try to match bass guitar or kick drum frequencies (which could lead to absurd settings like extreme bass boosts). If detection is confident that reference is unsuitable, we might even politely refuse or ask for a DI of the guitar if possible.

- **Risk: Different guitars/pickups cause mismatch between user's tone and reference-based suggestion.**
  Context: The assistant might craft a preset perfect for the reference assuming similar input, but if the user's guitar is drastically different (say reference was a Les Paul with humbuckers, user has a Strat single coil), the outcome can sound off (too thin or too boomy etc.).
  Mitigation: Introduce a **guitar calibration step** or user input for guitar type. For instance, in the UI setup, ask the user what type of pickup (single coil vs humbucker, active vs passive) or provide presets for "Guitar profile". Alternatively, have the user play a known reference riff on their guitar clean and analyze it. Determine a simple EQ profile of their guitar (like does it lack bass, or have a spike in highs). Then, during tone matching, incorporate that profile: we can pre-EQ or adjust the target to account for it. Example: if user's DI is much brighter than the one used to derive reference tone, the assistant could either instruct "roll off your tone knob slightly" or more practically, automatically add a slight EQ compensation when applying settings. Also, in our dataset/algorithm, incorporate knowledge of guitar differences: e.g., train/test with multiple guitars so the model learns to compensate.
  Additionally, if mismatch is noticed after suggestion (like user says "it's more fizzy than it should be"), we could have a quick *fine-tune knob* on UI: e.g., a "Guitar compensation" slider (more bright vs more dark) that essentially tweaks an EQ or the Presence globally. This way the user can quickly correct for their instrument.
  Ultimately, clear user communication: no algorithm can fully negate pickup differences – we'll clarify that some manual fine-tuning might be needed if guitars differ greatly. Encouraging the user to provide a reference recorded with the *same guitar* they're using (if possible) is another mitigation, as that inherently accounts for it.

- **Risk: Users chasing "impossible" matches (due to gear or physics limitations).**
  Examples: A user with a Strat asks for Meshuggah's 8-string tone – the guitar itself can't produce those low frequencies or tightness. Or someone expects a 5W practice amp sim to sound like a cranked tube stack recorded in a pro studio. There are also production elements (double tracking, precise post-processing) that our single-chain suggestion can't replicate.
  Mitigation: **Set expectations & educate.** The assistant's UI or documentation will note that it provides an approximation given the user's setup. If a user picks a target outside the realm of their gear, we could detect some of it (e.g., if user has indicated a 6-string standard tuning and target tone's fundamental is clearly down at F#0 from an 8-string, we might warn "Your guitar's tuning/pitch is higher than the reference's; results will approximate the tone but won't add those missing low frequencies"). If an exact amp model or effect is key to a tone but the user doesn't have it, we say so: "The target tone was originally made with a unique gear (e.g., a specific fuzz), we'll get close with what's available."
  We will also avoid over-promising in marketing: phrase it as a *"tone assistant to get you in the ballpark or slightly beyond"* rather than "clone any tone perfectly". In-app, perhaps have a tooltip on accuracy.
  Also, if after matching the user still isn't satisfied, provide guidance: maybe suggest they try a

different pickup, or layering. The assistant could have a help note like "This tone in the original recording was double-tracked; to achieve that fullness, try recording your part twice and pan left/right." – addressing the gap between what one guitar can do vs produced track.
In summary, be transparent and provide tips. Possibly incorporate a **knowledge base** for famous tones: e.g., if user asks for Van Halen "Brown Sound", mention "Note: The original tone uses an MXR phaser and tape echo; our preset covers the amp and EQ, but adding a slight phaser might help." This handles user expectations and fosters trust.

- **Risk: Copyright and Licensing Concerns.**
  Several angles:

- **Impulse Responses (IRs):** High-quality IRs of famous cabs/mics are often sold commercially. If our assistant effectively "tone matches" a reference and in doing so produces an IR (like a custom EQ curve), is that an issue? If we explicitly output an IR file that is essentially an EQ match of a copyrighted tone, that could be problematic if it's considered derivative of the original recording. However, if it's just a set of EQ adjustments, it's more like a user-created preset. To be safe, we will **avoid directly providing any IR that was generated purely by matching a copyrighted recording**. Instead, we confine ourselves to *existing IRs or combinations that are known free or licensed for use*. For example, if the target tone is known to be a Mesa cab with SM57, instead of deriving an IR from the song, we'll pick a Mesa/SM57 IR from our library (ensuring it's a free one or one we have rights to distribute) that yields similar sound. So the assistant doesn't produce an IR from thin air; it picks among IRs. If a perfect match would require a very custom IR, we either approximate or instruct user to do a manual refinement with an EQ themselves.
- We will also ensure any **IR files we include** (if any) are either user-supplied, from free packs, or our own captures. We will not include commercial IR content without permission.
- **Artist Names and Trademarks:** Referring to "Metallica" or specific song names might have trademark issues in a product. In the UI we might phrase things carefully (maybe "California '86 Thrash Rhythm" instead of explicitly "Metallica's Master of Puppets"). But since users will likely use those names, the assistant could interpret them internally. It's probably okay to mention artist names in a descriptive context (fair use as reference) but not to use their branding as endorsement. We won't include any actual audio from artists, just mimic gear settings.
- **Gear models naming:** Many amp modelers avoid actual trademark names (e.g., calling a Marshall JCM800 model "British 800" etc.). We should do the same in UI to avoid legal issues with trademarked amp names. But the user likely understands the alias.
- **Training data licensing:** If we use community presets or captures, need to ensure license. Neural Amp Modeler captures are user-shared freely typically, but we'll double-check any usage terms. Synthetic data we generate ourselves is fine. If using any portion of a song as reference in internal tests, we won't distribute that.

- **Solution:** Work primarily with *public domain or licensed data*, use generic descriptors for gear and tones, and when suggesting IRs or presets, stick to either user's own assets or free ones. Possibly allow user to import their IR collection and then we can pick from those (so if they own certain IR packs, we can leverage them without distributing anything).

- **Risk: Incorrect or Unsafe Recommendations (General AI errors).**
  Even outside the known categories, the algorithm might occasionally spit out a nonsensical suggestion due to a bug or edge case (e.g., recommending extremely odd settings that don't actually yield a good sound). This could frustrate users or, in worst case, harm equipment (less likely in

software, but say it maxed out output and someone's speakers are loud). *Mitigation:* We incorporate sanity checks in the pipeline:

- Constrain parameter ranges to sensible limits (we won't ever set output volume to 11 if the plugin goes to 11, we'll cap around unity gain).
- Use the evaluation features as a guard: e.g., after generating a preset, quickly simulate a short sound and see if there's anything clearly wrong (like the presence of extreme high-frequency noise meaning maybe a filter didn't apply – we can catch that and adjust or at least warn).
- Implement a "fail-safe preset": If the system cannot find a good match or gets confused, it's better to output a decent generic tone than something crazy. So we could detect if similarity score is below a threshold after optimization. If so, maybe abandon and load a fallback like "generic high-gain preset" and say "We had difficulty matching precisely, here's a basic starting tone you can refine." That prevents outputting junk.

- Beta testing will likely reveal any such odd recommendations which we can then address rule-based or via improving the model.

- **Risk: Performance/CPU usage too high.**
  If the user tries live mode on an older machine or does an offline match on a long audio clip, CPU or memory could spike. Amp sims are CPU heavy, and our optimization might instantiate multiple quick runs. If poorly managed, it might glitch audio or hang. *Mitigation:* We design optimization to use short segments of audio (we don't need to analyze a 5-minute song; 10 seconds of representative riff is enough). We'll explicitly limit how much audio is processed for matching – maybe the user selects or the system detects a loop of the riff in reference. Also, heavy computation should run on a worker thread so as not to stall real-time audio. For live usage, keep computations minimal and possibly at a lower priority. We also allow the user to cancel analysis if it's taking too long or they're unhappy (stop button). If memory is a concern (loading large IRs or models), we'll test on moderate spec systems and optimize (e.g., unload any large model from RAM when not needed).

- **Risk: User overload or misuse.**
  If the assistant is too complicated or provides too many options, users might be overwhelmed. Or some might misuse it (e.g., try to match a piano sound or something not intended). *Mitigation:* Keep UI simple for main workflow, hide advanced options behind an "Advanced" expander. Provide guidance in UI (like tooltips, a quick tutorial modal for first use). If misuse (like non-guitar input) is detected, just politely inform "This tool is designed for guitar tones; the input doesn't seem to be a guitar – results may not be valid." That way the user knows.

- **Risk: Competition with similar features (market risk).**
  If e.g. Positive Grid's built-in AI is very good, why would users use ours with BIAS? Or if Neural DSP releases their own assistant. *Mitigation:* Focus on integration and openness – we support multiple platforms, giving users flexibility, and aim for high accuracy and transparency (maybe our methods could yield more customizable results). Also highlight the educational aspect (how we explain changes). If our algorithm is truly effective, word will spread. But market risk is beyond purely technical – still, by designing it generally and not tying to one ecosystem, we reduce risk of being locked out.

In summary, we are aware of these risks and have incorporated specific measures – from technical fixes like gating and EQ checks to user communication and design choices – to manage them. Ongoing testing and user feedback will further inform adjustments to keep the system safe, legal, and user-friendly.

10. Decision Log (Knowns, Unknowns, Assumptions)

Key Decisions Made (Known):
- We will implement a **hybrid tone matching strategy**: using a curated library of tones for quick starting points, combined with an optimization algorithm for fine-tuning, and potentially a trained ML model for fast predictions. This decision was made to balance accuracy (optimization can achieve precise matches [11]) with speed (retrieval/model avoids long searches) and leverage domain knowledge (library encapsulates known good tones).
- The **signal chain** is standardized to Gate → Boost → Amp → Cab/IR → EQ. We assume this covers the essential components for heavy metal tones and decided not to include effects like reverb, delay, etc., in scope. This simplifies the problem to core tone.
- The system will operate as a **desktop solution**, likely a plugin (VST3/AU) that can host or interface with other amp sim plugins. This decision was based on user's preference (desktop, VST) and the need for tight integration with existing workflows. We are not pursuing a cloud service or mobile app at this time, focusing on local processing for latency and privacy.
- **Export strategy:** We will directly support exporting native preset files for a handful of popular amp sim platforms (Neural DSP, Line6 Helix Native, etc.), to seamlessly integrate with those tools. We chose this over only giving abstract suggestions because it significantly improves user convenience. We also decided to include an intermediate JSON format for extensibility.
- **Evaluation focus:** We decided to emphasize perceptual audio metrics and human testing. The success criteria are defined in terms of sound similarity and user preference, not just matching knob settings. This aligns with the ultimate goal (good sound) rather than any internal metric.
- **User in control:** A conscious decision was made to ensure the assistant is a *non-intrusive advisor*. All changes are preview-able and undoable by the user, addressing any concern of AI taking over their tone completely. This decision improves trust and adoption.

Unknowns and Open Questions:
Despite the detailed plan, some aspects remain uncertain and will require experimentation or further input:
- **Effectiveness of feature metrics**: Are the chosen audio features (spectral bands, transient measures, etc.) sufficiently capturing the perceptual differences in tone? This is partially known from audio science, but specific weighting for "what sounds closer" is somewhat unknown. We will likely need to tune the similarity metric weights by trial and error and possibly learning from listening tests (unknown optimal weights).
- **Optimization performance:** It's unknown how many iterations a method like CMA-ES will need for a typical tone match and if that can be done in, say, under 10 seconds of audio processing. We assume it's manageable with a good initialization, but real-world test will tell. If it turns out too slow, we may need to adjust strategy (e.g., rely more on model or simplify parameter space during search).
- **ML model generalization:** If we train a model to predict settings, will it generalize to real-world signals outside our training data? We assume yes for within distribution, but guitars and playing vary a lot. Unknown if the model might, for example, always overshoot presence for single coils or such. Only testing with various inputs will uncover this.
- **Preset format details:** Some plugin preset formats (like Bias or AmpliTube) we haven't reverse-engineered yet – it's unknown how complicated they are. We assume we can parse/generate them, but if any format is encrypted or proprietary, that could block direct export. If so, backup plan might be

instructive output or requiring user to load a preset via the plugin's own tools.
- **User adoption & behavior:** It's unknown how users will primarily use this tool – e.g., will most use reference audio matching vs text requests? We assume both are needed, but their frequency is unknown. Also, will users try to abuse it in unintended ways (like tone matching things that aren't guitar)? Possibly. We might need to adapt based on usage patterns discovered in beta (unknown until real users interact).
- **Integration with many plugins:** If hosting plugins internally, an unknown is how stable or feasible it is to support many third-party plugins in one host (some plugins might not like being loaded inside another plugin). We may need to limit or find specific technical solutions. This is an engineering unknown until we attempt multi-plugin hosting.
- **Live mode viability:** We're unsure how helpful or feasible the live assist will be in practice. It may turn out that trying to auto-adjust while playing is distracting or not accurate enough in real-time. We assumed a scaled-down approach for live, but its value needs prototype validation. Possibly we might pivot live assist to just visual metering rather than auto-changes if that works better (TBD).

Assumptions (that we'll verify):
- We assume **users have a fairly clean DI signal** (no weird EQ or clipping) and a typical audio interface. If user DI is very noisy or already processed (e.g., they accidentally feed an already distorted signal), the system might mis-operate. We might need to detect non-DI input. But our design assumes DI.
- We assume the **target tone is achievable** with the gear available. If a user doesn't have any high-gain amp sim, our suggestions might not be useful. We might assume they have at least one decent high-gain amp sim plugin installed. If not, our system can't pull magic – we might then suggest "you need a high-gain amp sim in your chain for this tone."
- We assume **tone can be matched by adjusting our defined parameters** and that the reference tone differences are mostly EQ/distortion differences. If the tone difference is something like "uses a chorus effect" or "double-tracked", that's outside our parameter set – we assume those aspects are minimal or can be ignored for matching core tone.
- We assume **our curated library covers the space of user queries** sufficiently that retrieval always provides a reasonable starting point. If the user requests something extremely niche not in library, worst case we fall back to generic optimization. We assume that's acceptable though maybe slower.
- We assume **license compliance**: by focusing on user's own plugins and free IRs, we won't hit legal issues. We will, however, double-check any instance where we use names or content.

Next Steps (Experiments to resolve unknowns):
To address the unknowns, we propose several near-term experiments and tasks: 1. **Feature Validation Experiment:** Take a few pairs of tones (slightly different knob settings), compute our feature differences and see if they correlate with audible differences. We might adjust our feature extraction accordingly (like realize we need to add a certain metric if something audible wasn't captured). 2. **Optimization Speed Test:** Implement a small prototype of parameter search (perhaps for a simple EQ match scenario first or using a simplified amp sim) and measure time vs accuracy. Try both CMA-ES and Bayesian to see which converges faster for this domain. This will guide the tuning of our optimization (population sizes, iteration limits). 3. **Library Retrieval Efficacy:** Build a tiny tone library (maybe 5 presets with distinct sounds) and do a test retrieval for intermediate target tones to see if it picks the closest logically. Also test text mapping: feed some example text queries and see if our tag system yields an appropriate preset (this could be done manually first). 4. **Model Training Feasibility:** Generate a small dataset (maybe 100 random presets) and train a quick regression model. Test it on a few unseen tones to gauge whether it gets in the ballpark. This will inform how large a dataset we need and if the approach is promising. 5. **Preset Export Trials:** Manually create a preset in one of the target plugins, then attempt to generate a file that duplicates it. See if the

plugin can load it and sound identical. This will expose any format quirks early on. 6. **Guitar Variation Test:** Try the system (or parts of it) with two very different guitars on the same target. For example, match "djent tone" with a Strat vs with an EMG pickup guitar and see the differences. If our approach yields two different recommended settings (it likely will, and should), verify that each indeed sounds best for that guitar. If one guitar consistently can't achieve it even with changes, we note where the limitation lies. 7. **GUI/Workflow Mock Testing:** Even before full implementation, do a paper or figma prototype of the UI and run through user stories with potential users. See if they understand how to operate it, or if instructions need clarifying. Better to catch UX confusion early. 8. **Edge-case audio tests:** Feed things like pure noise, very short staccato only, or very legato smooth playing to feature extraction to ensure it doesn't break (like dividing by zero in some measurement or mis-detecting silence as tone). 9. **Source separation trial:** Evaluate one or two publicly available guitar separation tools on a mixed song clip to see if it's worth integrating. If the quality is too poor (leaving a lot of drum bleed), we might choose to not attempt it and rely on user providing better input.

These experiments will reduce uncertainty and allow us to refine parameters before coding the full system.

MVP Scope and Prioritization:
For the **Minimum Viable Product (MVP)**, we will implement a vertical slice focusing on the highest-value scenario: *reference audio matching with one supported amp sim.* Specifically, MVP would likely be: - Support one plugin (say, Neural DSP Archetype or a high-quality free amp sim) for which we can do the full chain. That means building tone library entries for that sim, running our algorithm specifically with that sim's models. - The user can load a reference guitar clip, our system suggests settings (and possibly automatically applies them if we host that sim). - Preset export for that one plugin is supported (others can be added later). - Basic UI with the essential controls (load reference, suggest, A/B). Possibly text query if easy, but we might leave text for v1 if time is short, focusing on audio matching first (since that's more objective to validate). - We'll include at least one example IR or use plugin's own cabs to handle cab matching. - No live mode in MVP – only offline suggestion. - Evaluation: we'll test MVP on a few known tones to ensure it's working reasonably.

This MVP delivers the core promise (automatically dial a tone to match something) in a limited context, which we can then extend.

v1.0 (Full Feature Release) Potential Additions:
- Expand to multiple plugin targets and more library content. - Add the text-based query interface with a decent library of styles. - Implement the ML model for faster suggestions, if MVP relied mostly on retrieval/opt. - Polish the UI, add the explanation texts for changes, etc. - Include the preset export for all chosen platforms. - Ensure robust testing and performance improvements found in MVP stage.

v2.0 (Future Ideas):
- Real-time assist mode fully implemented, possibly with an always-on-tone shaping that nudges settings dynamically (needs careful design from MVP feedback). - A larger cloud-based tone library integration (user can download more tone references, share presets with community). - Possibly user profiling (let the assistant remember the user's guitar profile or preferences over time). - Extend beyond metal perhaps (though original scope is metal, v2 might broaden styles, but that's a business decision). - More advanced AI models (like integrating a deep learning metric for tone similarity learned from data, as in ST-ITO's AFx-Rep [9], to further improve matching beyond our handcrafted features).

Throughout, we will keep logging decisions and lessons. Each assumption validated or refuted will be recorded and the design adjusted accordingly.

In summary, we have a clear plan for MVP and iterative improvements, along with a log of what we know and what we need to find out. This closes the design sections. Below, we provide additional implementation guidance: architecture summary, the build roadmap, and example prompts for coding key components.

Architecture Overview (Diagram in Text)

Components and Data Flow: The AI Tone Assistant is composed of several interconnected components:

- **User Interface (UI)** – Provides controls for users to input their desires (reference audio file or text description, etc.) and to trigger the tone suggestion process. It also displays the suggested tone settings and allows A/B comparisons. The UI is either a standalone window or a plugin UI in a DAW.
- **Tone Analysis Engine** – The core logic that runs in the background. It includes:
- *Feature Extractor:* Takes audio input (reference clip, user's DI) and computes the audio feature set (spectral bands, centroid, transients, etc.).
- *Tone Matcher:* Implements the hybrid strategy – it queries the Tone Library (below) for nearest presets if applicable, possibly uses the ML Prediction Model for an initial guess, and then runs the Optimization module to fine-tune parameters to best match the target features. The Tone Matcher produces an output TonePreset (an abstract representation of the recommended chain and settings).
- *ML Prediction Model (optional in first version):* A trained model that can predict a TonePreset from target features quickly. If available, Tone Matcher uses it to shortcut some of the search. Otherwise, Tone Matcher relies on library + optimization.
- *Tone Library:* A database of example TonePresets with their associated feature vectors and metadata (e.g., tags like "Metallica tone"). This can be a JSON or small database loaded at runtime. The Matcher queries it for similar tones or for text search.
- *Optimization Module:* Implements parameter search (CMA-ES / Bayesian) and uses the Feature Extractor and a similarity evaluator to iterate towards a better TonePreset match. This module will loop: tweak TonePreset, run a simulation (passing DI through simulated chain) to get resulting features, compare to target, and keep the best.
- *Audio Simulation (DSP) Interface:* This is how the Tone Matcher tests a TonePreset – by actually applying those settings to either a real plugin or an internal model to get audio output for feature comparison. If the assistant hosts an amp sim, this interface controls that plugin's parameters and processes the audio buffer. If we have differentiable models, could use them here too.
- **TonePreset Schema & Mapper:** The standardized representation of a preset. The Mapper part is responsible for translating a TonePreset into specific plugin parameters or file formats. It contains mapping tables for each supported plugin (e.g., which internal amp corresponds to TonePreset's "5150").
- **Plugin Host / Integration Layer:** If the assistant runs as a plugin, this layer either hosts the amp sim plugin (embedding it) or communicates with the DAW/host environment to insert the suggested settings. It also handles capturing the user's DI from the host and outputting the processed audio. If standalone, this layer might simply load audio files and use an internal DSP chain for known amps.
- **Preset Exporter:** Uses the TonePreset and mapping info to generate preset files (.preset, .hlx, etc.) for the chosen target plugin. This may involve writing JSON/XML or binary files in the correct format and packaging IR files if needed.

- **Data Storage:** Files on disk include the Tone Library data, any machine learning model files, and possibly a cache of previous analysis results. Also stored are any included IR files or user-imported IR libraries.

Flow Example (Reference Audio to Suggested Preset):
1. User selects a reference audio file (or chooses an artist name from a dropdown) and clicks "Analyze".
2. The UI signals the Tone Analysis Engine. The Feature Extractor reads the reference audio and computes the target tone features. If user also provided a DI of their playing, features from that (or at least loudness calibration) are computed too.
3. The Tone Matcher takes the target features: - It queries the Tone Library for nearest matches (in feature space if audio, or via tags if text). Suppose it finds a preset that's pretty close.
- It may also run the ML Prediction Model with these features to get another candidate preset.
- It then evaluates these candidates by simulating audio (taking a short segment of the user's DI, processing through the candidate settings, extracting features). The best candidate becomes the starting point.
- The Optimization Module then fine-tunes: it perturbs parameters (e.g., adjust EQ knobs slightly, try alternate IR from a small list) and uses the simulation loop to see if similarity improves. After several iterations or convergence, it ends up with a final TonePreset that scores highest on similarity.
4. The resulting TonePreset is returned to the UI.
5. The UI, using the Plugin Host/Integration, applies the TonePreset so the user can hear it. If our assistant plugin was hosting the amp sim, it now sets all the parameters on that amp sim instance to match TonePreset. If standalone, perhaps it loads an internal audio demo or instructs the user to manually load the preset. In our design, likely the user can hear it directly if it's all in one plugin chain.
6. The UI allows the user to toggle between their original tone (if they had one dialed) and the new suggestion. It also displays the settings (maybe a GUI representation of the amp with knob positions, etc.). The user can make minor tweaks if desired.
7. If user is satisfied, they click "Export Preset" and choose their plugin (if not already known). The Preset Exporter then generates the preset file and saves it. The user can then load that in the actual plugin software if needed (if we weren't already controlling it). If the assistant itself was hosting the tone, the user might not need an export unless they want to use that tone outside this context.
8. If the user requests a different target or adjustment, they can go back or refine the text prompt etc., and the process repeats.

Real-time Assist Flow (if active):
- The Feature Extractor continuously monitors the incoming DI and output. If it detects, for example, increased noise or changed picking style, it might trigger a quick re-evaluation.
- The ML Prediction Model could be used frame-by-frame to adjust one or two parameters gradually (like a slow adaptive EQ).
- Alternatively, it just collects metrics and flashes a suggestion (like a notification: "Consider raising your Noise Gate threshold"). Implementation might be simpler as advisory rather than full auto-change in live mode.

Architecture Emphasis:
We have clear separation of concerns: - UI/UX vs Analysis Engine vs DSP integration vs Data storage. This ensures each can be modified or replaced (for instance, in a different environment like a command-line version for batch processing, one could reuse the analysis engine without UI).
- The text "diagram" mapping: imagine UI at top sending inputs down to analysis; analysis consulting library and model (on the side), running optimization loop (maybe depicted as a cycle) feeding into a plugin DSP node; then result flows back up to UI for presentation and to exporter for output.

- The assistant architecture also needs to handle concurrency: audio thread vs analysis thread. We'll likely do analysis offline (when user hits suggest, perhaps the audio stops or we explicitly take a snapshot of DI). For live, a simplified continuous thread.

This architecture is designed for **extensibility** (new plugins support by adding mapping, new tone styles by adding to library, improved algorithms by swapping out model or optimizer) and **maintainability** (clearly defined modules like FeatureExtractor, PresetMapper, etc.). Each module can be unit-tested (e.g., feed synthetic audio to FeatureExtractor and verify output).

Now, with the architecture clarified, we proceed to the build plan and specific implementation steps.

Implementation Roadmap (Step-by-Step Build Plan)

We will build the AI Tone Assistant in incremental stages:

Phase 0: Research & Data Preparation
- *0.1.* Compile a small initial **Tone Library dataset**. Collect a handful of example tones: e.g., create 5-10 presets manually (or from community sources if allowed) covering popular metal tones (thrash, djent, etc.) in one plugin (say we choose Neural DSP Fortin Cali or a high-gain freeware amp sim for initial tests). For each preset, render a short DI clip through it and store the audio and preset. This will be used to test feature extraction and similarity.
- *0.2.* Develop the initial **Feature Extraction code** (likely in Python for ease). Input an audio file, output the feature vector. Test it on the example tones: verify it yields reasonable values (e.g., a scooped tone indeed shows lower mid-band vs high-band, etc.). Adjust feature set or normalization as needed.
- *0.3.* Decide on which **plugin or amp sim to use for MVP prototyping**. Possibly pick an open source one (like Ignite Amps Emissary or even a simple custom wave-shaping chain) to avoid licensing issues in dev. Later integrate commercial plugin via user's installation.
- *0.4.* If using a plugin, experiment with a **plugin hosting approach** in a simple form: write a small program to load a VST (maybe via JUCE or Steinberg SDK) and automate one parameter change to ensure we can programmatically control it and render audio offline. This is to de-risk the technical challenge of automating plugin DSP. Alternatively, build a simple amp sim in code to use as stand-in (like a fixed waveshaper + EQ to mimic an amp).

Phase 1: Core Matching Engine (Console Prototype)
- *1.1.* Implement a prototype Tone Matcher in Python: Given a target audio (and optionally one candidate preset), run a simple optimization loop on a simplified parameter set. For start, maybe optimize a 3-band EQ to match a frequency response of reference. This tests the optimization module (CMA-ES or so) in a simple scenario. Evaluate speed and result (should basically recreate an EQ curve like Fractal tone match does).
- *1.2.* Implement the **retrieval step**: using the mini tone library, for a new reference audio, compute features and find nearest neighbor. See what it picks. This is just a k-NN in feature space, straightforward.
- *1.3.* Combine retrieval + simple optimization: e.g., take the preset from retrieval as starting point, then allow the optimization to tweak 2-3 parameters around it to better match. Test with one of the example references that wasn't in library to see improvement.
- *1.4.* Expand the TonePreset schema and incorporate the **mapping logic** for our chosen test plugin. For now, maybe internal representation = external since we might be using one known plugin. But still, define the data class for TonePreset with fields. Write code to apply a TonePreset to the test plugin or sim (set

knobs accordingly).
- *1.5.* End of Phase 1: Have a script where you input a reference audio path, it outputs a TonePreset (and maybe writes a rudimentary preset file or prints settings). Try this with a few references to gauge quality.
- *1.6.* Iterate on feature weighting or algorithm as needed if results are not close.

Phase 2: MVP Application (Single plugin, audio matching)
- *2.1.* Start implementing the actual application structure. Possibly use JUCE (C++): set up a basic audio plugin project. Implement the UI with a file picker for reference and a button "Suggest Tone". (If not comfortable with JUCE UI yet, could start with a simple CLI or minimal GUI in Python just to orchestrate, then port UI to C++ later.)
- *2.2.* Integrate the Python analysis engine with the UI. E.g., the UI when clicked could call a Python script with subprocess passing the reference path, which returns a TonePreset (perhaps as JSON). Alternatively, consider embedding Python or using PyBind if going C++ and Python mix. As MVP, even if it's a bit hacky (like writing to a temp file), it's okay. The goal: user can select audio, press go, and after some seconds see a result in UI.
- *2.3.* Implement audio playback in UI: allow user to audition the result. This might require hosting the amp sim plugin with the new settings. Possibly easier for MVP: after suggestion, just show the knob settings and allow user to manually dial them in their plugin to hear (which is not ideal). But maybe we can at least apply changes if our plugin is in the chain. If using JUCE, we can load the amp sim in our plugin's processing block and update parameters. So set that up: in the AudioProcessor, have a `AudioPluginInstance` for the amp sim loaded (hard-code the plugin path or use plugin scanning if possible). Then, when TonePreset comes back, map it to that instance's parameters.
- *2.4.* Test the end-to-end MVP: Insert our assistant plugin on a track with DI recorded. Load a reference file of a tone. Click suggest, wait, then the amp sim inside our plugin is set to new tone. Play the track (or live input) to hear the result. A/B can be done by bypassing our plugin vs enabling it (if bypass means original DI goes to maybe a different chain). For MVP, A/B might be manual if needed (like user disables our plugin vs enables to compare).
- *2.5.* Evaluate the result quality with a couple of references (some might be good, some not). Identify any glaring issues, fix in code (like if fizz too high, maybe add a default post-EQ cut in suggestion logic as a quick fix or refine features).
- *2.6.* Clean up MVP: ensure no memory leaks, stable performance for a single suggestion. Document usage.

Phase 3: Extended Functionality (Multiple gear, UI polish, text input)
- *3.1.* Add support for additional **amp sim models or plugins**. E.g., integrate Helix Native if available or another free VST. Expand TonePreset mapping to differentiate amp choices. Possibly incorporate switching amp model in suggestions (the optimization can now also try different amp if initial guess seems off). This is more complex because a different amp model means a different parameter set. But we can handle it by including a discrete choice in optimization (maybe try library presets of multiple amp types and pick best).
- *3.2.* Implement the **text query** interface: maybe a dropdown of known styles or an input box with autocomplete of known artists. Use the Tone Library metadata to find a matching entry or combination. For MVP, this can be simple (choose a preset by name). Later, integrate a more flexible interpretation (e.g., split user text into keywords and match tags).
- *3.3.* UI enhancements: Add an A/B button that stores the current DAW chain state before applying suggestion (or simply toggles the amp sim between old and new parameters). If our plugin hosts the amp, we can keep two sets of parameters and swap. Or we could run two amp instances (one with original, one with suggestion) and crossfade. This might be heavy but ensures instant A/B. For now, maybe a simpler approach: before applying new tone, we store original preset to restore on A/B toggle.

- *3.4.* Implement user adjustable options: e.g., allow user to pick which amp sim to use for suggestion (if they have multiple). Provide a dropdown of available targets. This will tell the system which mapping and which internal plugin to load.
- *3.5.* Preset Export: Now implement writing actual preset files for each target plugin supported. Use examples and documentation to format correctly. Test each by loading into the actual plugin software. This might require dealing with base64 or checksums depending on format. Focus on a small number first and ensure correctness.
- *3.6.* Incorporate the trained **ML model** if we have one. Possibly train a better one now with more synthetic data (maybe generate a few thousand random presets offline and train on that). Integrate it in C++ (convert to ONNX or use libtorch). Use it to speed up suggestions: e.g., get an initial guess without heavy library search. Evaluate if it improves overall speed and whether quality remains good.
- *3.7.* Live Assist Mode: Implement a simple version as experimental – perhaps a mode where the plugin monitors input loudness and noise and auto-adjusts gate threshold in real time (a simple case). This can demonstrate the concept without tackling full dynamic re-tuning. Or implement a refresh button that re-analyzes continuously every few seconds if live mode is on. Up to design, but keep it minimal for now.
- *3.8.* Expand Tone Library to more entries and refine tag data. Possibly allow an external file so that updates can be made without recompiling (maybe a JSON in resources that we can edit to add new tone references).
- *3.9.* Housekeeping: Add comprehensive error handling (if analysis fails, or plugin not found, give user message instead of crash). Also implement the caching mechanism: e.g., after a suggestion, cache the (reference fingerprint -> TonePreset result) so subsequent identical requests return immediately. This could be as simple as a dictionary stored for session (persist if needed).

Phase 4: Testing & Refinement
- *4.1.* Create the evaluation set and run offline tests. Compute similarity metrics for a set of test references, see if within acceptable range. Tweak the algorithm if some metric consistently off.
- *4.2.* Conduct small user testing: give the tool to a few guitarists to try. Collect feedback on UI and tone results. For example, they might say “The suggested tone for X was close but too fizzy” – then we know to adjust fizz control. Or “the UI was confusing on how to load my plugin” – fix documentation or UI flow.
- *4.3.* Implement any quick improvements from feedback: e.g., maybe add a “fine-tune brightness” slider for user to adjust output if they feel it’s needed. Or if someone had trouble with references that are too short, maybe enforce a minimum length or loop it.
- *4.4.* Regression tests: ensure that if we change something in algorithm (like weightings), previous test cases that were good don’t become bad. Use version control to manage these tweaks. Possibly maintain multiple algorithm presets (maybe user can choose between e.g. “fast mode” vs “accurate mode” if we find different parameter sets).

Phase 5: Documentation and Release Prep
- *5.1.* Write user documentation (or at least help tooltips) explaining how to use the assistant, what to expect, and known limitations (like requiring isolated guitar for best results, etc.).
- *5.2.* Ensure licensing is sorted: confirm that we’re not including any disallowed content, that our use of plugin control is within terms, etc. If needed, remove or replace anything questionable.
- *5.3.* Finalize packaging: if it’s a plugin, build installers for Mac/Windows, including the model file, library file, etc. Test installation on fresh systems.
- *5.4.* Mark experimental features (like live mode, if not fully vetted) as beta in the UI so users know it might not be perfect. Possibly behind a toggle in settings.
- *5.5.* Release MVP/v1.0 to broader users, then monitor feedback for further improvements.

This roadmap ensures a functional core is delivered early (phase 2 MVP), and then layers on the more advanced or extensive features. It allows testing the hardest parts (analysis, plugin control) first in a controlled way, then broadening scope. Each phase yields a working product increment that can be validated.

Now, to facilitate development, we outline some *Codex/AI code generation prompts* for key components in the next section, which could speed up implementation of certain pieces by generating boilerplate or routine code.

Codex Prompts for Implementation

Below is a set of suggested prompts that could be given to an AI code generation tool (like OpenAI Codex) to help generate implementations for various parts of the system. Each prompt is designed to produce code or configuration for a specific component based on the design above. These prompts assume the developer will review and integrate the generated code appropriately.

1. TonePreset Schema & Mapping System
This prompt asks Codex to create a data structure for TonePreset and functions to map it to a specific plugin's format. We include a small mapping table example for one plugin.

```
# TonePreset Schema and Mapping
# We need a data class to represent a guitar tone preset with components: noise
gate, pedal, amp, cab, and post-EQ.
# Each component has certain parameters. Provide a Python class TonePreset with
nested structures or attributes for each component.
# Include fields: gate_threshold (float), gate_release (float), pedal_model
(str), pedal_drive (float), pedal_tone (float), pedal_level (float),
pedal_enabled (bool),
# amp_model (str), amp_gain (float), amp_bass (float), amp_mid (float),
amp_treble (float), amp_presence (float), amp_depth (float), amp_master (float),
# cab_ir (str), cab_metadata (dict for cab type/mic info),
# post_eq (list of dicts for EQ bands, each with freq, gain, Q).
# Also implement a function to map TonePreset to a specific plugin preset
dictionary.
# For example, implement mapping for "NeuralDSP_FortinCali" plugin:
# - pedal_model "TubeScreamer" -> plugin pedal id "TS808",
# - amp_model "5150" -> plugin amp id "CaliIII",
# and map normalized 0-1 knob values to plugin's 0-10 scale.
from dataclasses import dataclass, field
from typing import List, Dict

@dataclass
class TonePreset:
    # Noise gate
    gate_threshold: float = -60.0  # dB
    gate_release: float = 100.0   # ms
```

```
    # Pedal (boost/OD)
    pedal_model: str = "None"
    pedal_drive: float = 0.0
    pedal_tone: float = 0.5
    pedal_level: float = 1.0
    pedal_enabled: bool = False
    # Amplifier
    amp_model: str = "UnknownAmp"
    amp_gain: float = 0.5
    amp_bass: float = 0.5
    amp_mid: float = 0.5
    amp_treble: float = 0.5
    amp_presence: float = 0.5
    amp_depth: float = 0.5
    amp_master: float = 0.5
    # Cabinet / IR
    cab_ir: str = ""
    cab_metadata: Dict[str, str] = field(default_factory=dict)  # e.g., {"cab":
"Mesa412", "mic": "SM57"}
    # Post-EQ
    post_eq: List[Dict[str, float]] = field(default_factory=list)  # list of
{"freq":..., "gain":..., "Q":...}

def map_to_neural_fortincali(preset: TonePreset) -> Dict:
    """
    Map TonePreset to a Neural DSP Fortin Cali preset structure.
    Returns a dictionary that could be serialized to the Fortin Cali preset
format (simplified for this example).
    """
    plugin_preset = {}
    # Map noise gate
    plugin_preset["GateThreshold"] = preset.gate_threshold
    plugin_preset["GateRelease"] = preset.gate_release
    # Map pedal (assuming Fortin Cali has a TS808 pedal with parameters Drive,
Tone, Level)
    if preset.pedal_enabled and preset.pedal_model.lower() in ["tubescreamer",
"ts9", "808"]:
        plugin_preset["Pedal"] = {
            "type": "TS808",
            "drive": preset.pedal_drive * 10.0,   # 0-1 normalized to 0-10
            "tone": preset.pedal_tone * 10.0,
            "level": preset.pedal_level * 10.0,
            "enabled": True
        }
    else:
        plugin_preset["Pedal"] = {"type": "TS808", "enabled": False}
    # Map amp settings (amp_model mapping)
    amp_type_map = {
```

```
        "5150": "CaliIII",   # hypothetical mapping: our "5150" corresponds to
Fortin Cali's Cali III amp model
        "JCM800": "Brit800"
    }
    amp_type = amp_type_map.get(preset.amp_model, preset.amp_model)
    plugin_preset["Amp"] = {
        "type": amp_type,
        "gain": preset.amp_gain * 10.0,
        "bass": preset.amp_bass * 10.0,
        "mid": preset.amp_mid * 10.0,
        "treble": preset.amp_treble * 10.0,
        "presence": preset.amp_presence * 10.0,
        "depth": preset.amp_depth * 10.0,
        "master": preset.amp_master * 10.0
    }
    # Map cabinet/IR
    if preset.cab_ir:
        plugin_preset["CabIR"] = {"file": preset.cab_ir}
    else:
        # use metadata to choose cab if no specific IR file
        cab = preset.cab_metadata.get("cab", "")
        mic = preset.cab_metadata.get("mic", "")
        plugin_preset["Cabinet"] = {"cab": cab, "mic": mic}
    # Map post-EQ (if any)
    if preset.post_eq:
        plugin_preset["PostEQ"] = [{"freq": band["freq"], "gain": band["gain"],
"Q": band.get("Q", 1.0)} for band in preset.post_eq]
    else:
        plugin_preset["PostEQ"] = []
    return plugin_preset

# Example usage:
preset = TonePreset(pedal_model="TubeScreamer", pedal_drive=0.2, pedal_tone=0.6,
pedal_level=0.8, pedal_enabled=True,
                    amp_model="5150", amp_gain=0.8, amp_bass=0.4, amp_mid=0.5,
amp_treble=0.6, amp_presence=0.7,
                    cab_ir="MesaV30_SM57.wav")
mapped = map_to_neural_fortincali(preset)
print(mapped)
```

This prompt will generate a Python dataclass `TonePreset` and a mapping function for a specific plugin, demonstrating how to convert normalized values to actual plugin param ranges and map model names.

2. Audio Feature Extraction Pipeline
Prompt to generate code for computing features from audio (maybe using `librosa` or similar in Python). Ensuring it computes spectral centroid, band energies, etc.

```python
# Audio Feature Extraction
# Implement a function analyze_guitar_tone(audio_samples: np.ndarray,
sample_rate: int) -> dict
# It should compute features: spectral centroid, band energy ratios (low, mid,
high, fizz),
# spectral tilt (slope of spectrum), RMS and crest factor (for dynamics),
# and an estimate of noise floor (maybe average energy during silent parts).
# Use librosa for spectral analysis.
import numpy as np
import librosa

def analyze_guitar_tone(audio_samples: np.ndarray, sample_rate: int) -> dict:
    features = {}
    # Ensure audio is mono
    if audio_samples.ndim > 1:
        audio = librosa.to_mono(audio_samples)
    else:
        audio = audio_samples
    # Normalize or at least compute overall level
    features["RMS"] = np.sqrt(np.mean(audio**2) + 1e-12)
    # Compute spectral centroid (mean frequency)
    centroid = librosa.feature.spectral_centroid(y=audio, sr=sample_rate)
    features["spectral_centroid_hz"] = float(np.mean(centroid))
    # Compute band energy ratios: define bands (in Hz)
    bands = {
        "low": (20, 250),
        "mid": (250, 2000),
        "high": (2000, 8000),
        "fizz": (8000, 12000)
    }
    # Compute power spectrum (welch or via STFT)
    S = np.abs(librosa.stft(audio, n_fft=4096))**2
    freqs = librosa.fft_frequencies(sr=sample_rate, n_fft=4096)
    psd = np.mean(S, axis=1)
    total_power = np.sum(psd)
    for band_name, (low_f, high_f) in bands.items():
        band_mask = (freqs >= low_f) & (freqs < high_f)
        band_power = np.sum(psd[band_mask])
        features[f"band_{band_name}_ratio"] = float(band_power / (total_power +
1e-9))
    # Spectral tilt: fit a line to the log-spectrum
    freq_mask = (freqs >= 50) & (freqs <= 10000)
    log_freqs = np.log10(freqs[freq_mask]+1e-9)
    log_psd = np.log10(psd[freq_mask] + 1e-12)
    if len(log_freqs) > 0:
        # linear regression for slope
        A = np.vstack([log_freqs, np.ones_like(log_freqs)]).T
```

```
        m, c = np.linalg.lstsq(A, log_psd, rcond=None)[0]
        features["spectral_tilt"] = float(m)
    else:
        features["spectral_tilt"] = 0.0
    # Crest factor (peak/RMS)
    peak = np.max(np.abs(audio))
    crest = peak / (features["RMS"] + 1e-9)
    features["crest_factor"] = float(crest)
    # Noise floor estimate: find silent sections (below -60 dB of max)
    threshold = peak * (10**(-60/20))
    if np.any(np.abs(audio) < threshold):
        silent_parts = audio[np.abs(audio) < threshold]
        if len(silent_parts) > 0:
            noise_rms = np.sqrt(np.mean(silent_parts**2) + 1e-12)
        else:
            noise_rms = 0.0
    else:
        # no truly silent part detected, estimate small
        noise_rms = features["RMS"]
    features["noise_floor"] = float(noise_rms)
    return features

# Example usage:
# audio_samples, sr = librosa.load("guitar_tone_example.wav", sr=None)
# feats = analyze_guitar_tone(audio_samples, sr)
# print(feats)
```

This prompt generates a Python function for feature analysis. It demonstrates how to compute various features, using librosa for spectral centroid, STFT for band energies, etc.

3. Recommendation Engine (Retrieval + Optimization)
Prompt Codex to code a simplified recommendation engine: maybe it uses a dummy library, finds nearest neighbor, then runs a basic iterative improvement loop adjusting one parameter at a time to minimize a fake "distance". We might not get full CMA-ES from Codex easily, but a conceptual example.

```
# Recommendation Engine (Retrieval + Simple Optimization)
# We have:
# - a list of known presets (library) with precomputed feature vectors in
library_features list.
# - a target_features dict from analyze_guitar_tone.
# Implement a function suggest_tone(target_features: dict) -> TonePreset that:
# 1. finds the nearest preset from library_features (Euclidean distance on
selected feature keys)
# 2. then does a simple iterative optimization: try small adjustments to the
preset's amp EQ (bass, mid, treble)
#    to see if it improves spectral match (like reduce difference in
```

```
band_mid_ratio, etc.).
# We'll assume library_presets (list of TonePreset) and library_feature_list
(list of feature dicts) are global or accessible.
import math

# Dummy global library for demonstration
library_presets = []
library_feature_list = []

def distance(feat1: dict, feat2: dict, keys=None):
    if keys is None:
        keys =
["band_low_ratio","band_mid_ratio","band_high_ratio","band_fizz_ratio","spectral_centroid_hz"]
    d = 0.0
    for k in keys:
        d += (feat1.get(k,0) - feat2.get(k,0))**2
    return math.sqrt(d)

def suggest_tone(target_features: dict) -> TonePreset:
    # 1. Retrieval: find closest preset in library
    best_idx = None
    best_dist = float("inf")
    for i, lib_feat in enumerate(library_feature_list):
        dist = distance(target_features, lib_feat)
        if dist < best_dist:
            best_dist = dist
            best_idx = i
    if best_idx is None:
        # if library empty, return a default TonePreset
        base_preset = TonePreset()
    else:
        base_preset = library_presets[best_idx]
    # 2. Simple optimization: adjust bass, mid, treble by small increments to
minimize difference in band energies
    best_preset = base_preset
    best_score = distance(target_features,
analyze_guitar_tone(simulate_audio(best_preset), sample_rate=44100))
    # (simulate_audio is a placeholder for processing DI through preset and
analyzing, assume it's available)
    # We'll simulate optimization by iterating a few steps over bass/mid/treble
    steps = [ -0.1, 0.1 ]
    for param in ["amp_bass", "amp_mid", "amp_treble"]:
        for step in steps:
            new_preset = TonePreset(**vars(best_preset))
            # apply step with bounds 0.0 to 1.0
            val = getattr(best_preset, param) + step
            val = min(1.0, max(0.0, val))
            setattr(new_preset, param, val)
```

```
            # evaluate new preset
            new_feat = analyze_guitar_tone(simulate_audio(new_preset),
sample_rate=44100)
            new_score = distance(target_features, new_feat)
            if new_score < best_score:
                best_score = new_score
                best_preset = new_preset
    return best_preset

# Note: simulate_audio is not implemented here (would require actual DSP
processing of DI through TonePreset).
# This pseudocode shows the structure of using retrieval and then iterative
parameter tweaks.
```

This code cannot fully run without `simulate_audio` (which would involve DSP), but it outlines how retrieval and an iterative parameter tweak loop might be structured. It selects nearest preset then tries adjustments.

4. UI Components & Flows
Prompt to generate a minimal PyQt or Tkinter UI for demonstration (just to illustrate). Actually, since final integration is more C++ plugin, maybe skip heavy UI code generation. But we could do a CLI or a pseudo UI logic: Maybe skip codex for UI, as it's mostly manual design with frameworks.

5. Preset Export Tooling
Prompt to generate, for example, a function to write a Helix preset JSON from TonePreset.

```
# Preset Export for Helix Native (JSON format)
# Implement a function to convert TonePreset to a Helix preset JSON structure.
# Helix presets have a structure with blocks, e.g., an amp block, cab block,
etc., each with specific parameters and values.
# We'll create a simplified version.
import json

def tonepreset_to_helix_json(preset: TonePreset) -> str:
    helix_preset = {
        "meta": {"product": "Helix Native", "version": "3.50"},
        "blocks": []
    }
    # Noise gate block
    helix_preset["blocks"].append({
        "block": "NoiseGate",
        "params": {
            "Thresh": preset.gate_threshold,
            "Decay": preset.gate_release
        }
    })
```

```
    # Pedal block (if enabled)
    if preset.pedal_enabled:
        helix_preset["blocks"].append({
            "block": preset.pedal_model,  # assume Helix has same name model
            "params": {
                "Drive": round(preset.pedal_drive * 10, 2),
                "Tone": round(preset.pedal_tone * 10, 2),
                "Level": round(preset.pedal_level * 10, 2)
            }
        })
    # Amp block
    helix_amp_map = {
        "5150": "Cali IV Lead",  # example mapping to Helix internal name
        "JCM800": "Brit 2204"
    }
    amp_block = {
        "block": helix_amp_map.get(preset.amp_model, preset.amp_model),
        "params": {
            "Drive": round(preset.amp_gain * 10, 2),
            "Bass": round(preset.amp_bass * 10, 2),
            "Mid": round(preset.amp_mid * 10, 2),
            "Treble": round(preset.amp_treble * 10, 2),
            "Presence": round(preset.amp_presence * 10, 2),
            "Resonance": round(preset.amp_depth * 10, 2),
            "Master": round(preset.amp_master * 10, 2)
        }
    }
    helix_preset["blocks"].append(amp_block)
    # Cab/IR block
    if preset.cab_ir:
        helix_preset["blocks"].append({
            "block": "IR",
            "params": {
                "File": preset.cab_ir,
                "LowCut": 80.0,
                "HighCut": 12000.0
            }
        })
    else:
        cab_name = preset.cab_metadata.get("cab", "4x12 Cab")
        mic_name = preset.cab_metadata.get("mic", "SM57")
        helix_preset["blocks"].append({
            "block": "Cab",
            "params": {
                "CabModel": cab_name,
                "Mic": mic_name,
                "LowCut": 80.0,
                "HighCut": 12000.0
```

```
            }
        })
    # Post EQ block if any bands
    if preset.post_eq:
        eq_params = {}
        # Helix parametric EQ has maybe 3 bands or 5 bands depending on model.
We'll just output first 3.
        for i, band in enumerate(preset.post_eq[:3], start=1):
            eq_params[f"Freq{i}"] = band["freq"]
            eq_params[f"Gain{i}"] = band["gain"]
            eq_params[f"Q{i}"] = band.get("Q", 1.0)
        helix_preset["blocks"].append({
            "block": "ParametricEQ",
            "params": eq_params
        })
    # Convert to JSON string
    return json.dumps(helix_preset, indent=2)

# Example usage
# print(tonepreset_to_helix_json(preset))
```

This prompt generates a JSON preset creation which can be saved as a .hlx file for Helix. It's simplified but shows structure.

6. Tests (Audio + Regression)
We can prompt Codex to create a small test scenario: maybe generating a synthetic tone and checking that when matched by the system, certain metrics are below threshold.

```
# Regression Test Example
# We want to test that the tone assistant does not produce an overly bright
"ice-pick" tone.
# Given a reference with moderate high-frequency content, verify the suggested
preset's high-band energy isn't too high.
# Pseudo-code:
import numpy as np

def test_no_ice_pick(di_signal, reference_tone):
    # di_signal: numpy array of DI audio
    # reference_tone: audio array of target tone (or features of target)
    target_features = analyze_guitar_tone(reference_tone, sample_rate=44100)
    suggested_preset = suggest_tone(target_features)
    # simulate audio for suggested preset (this requires actual DSP; assume we
have simulate_audio function)
    output_audio = simulate_audio(suggested_preset, di_signal)
    output_features = analyze_guitar_tone(output_audio, sample_rate=44100)
    # Check that high band ratio is not significantly higher than reference's
```

```
high band
    ref_high = target_features.get("band_high_ratio", 0)
    out_high = output_features.get("band_high_ratio", 0)
    ref_fizz = target_features.get("band_fizz_ratio", 0)
    out_fizz = output_features.get("band_fizz_ratio", 0)
    # Assert conditions
    assert out_high <= ref_high + 0.05, f"High frequencies too pronounced:
{out_high} vs {ref_high}"
    assert out_fizz <= ref_fizz + 0.05, f"Fizz frequencies too high: {out_fizz}
vs {ref_fizz}"
    # Also ensure loudness is similar
    ref_rms = target_features.get("RMS", 0)
    out_rms = output_features.get("RMS", 0)
    assert out_rms <= ref_rms * 1.1,
f"Output louder than reference by more than 10% (ref {ref_rms}, out {out_rms})"

# This test would be called with specific DI and reference samples.
```

This prompt outlines how a regression test might check for “no ice pick” by comparing band ratios. It includes assertions that can be part of an automated test suite.

These Codex-generated code blocks provide a starting point for implementing critical pieces. The developer should integrate them carefully (especially the placeholders like `simulate_audio` which in reality would interface with DSP). They illustrate how to structure the code according to our design.

---

1 2 3 5 Positive Grid BIAS X Review: AI Powered Assistant or a Serious Tone Machine?
https://www.kvraudio.com/positive-grid-bias-x-review-ai-powered-gimmick-or-a-serious-tone-machine

4 Positive Grid BIAS X Amp and FX Modeling Plug-in | Sweetwater
https://www.sweetwater.com/store/detail/BIASX--positive-grid-bias-by-amp-and-fx-modeling-plug-in

6 7 Tone Match block - Fractal Audio Wiki
https://wiki.fractalaudio.com/wiki/index.php?title=Tone_Match_block

8 Spectral Descriptors - MATLAB & Simulink
https://www.mathworks.com/help/audio/ug/spectral-descriptors.html

9 10 11 12 13 ST-ITO: Controlling audio effects for style transfer with inference-time optimization
https://arxiv.org/html/2410.21233v1

14 15 arxiv.org
https://arxiv.org/pdf/2411.14972